TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN



Các công nghệ lập trình hiện đại

Tìm hiểu LlamaIndex và xây dựng hệ thống hỏi đáp RAG về Quy chế đào tạo Trường Đại học Sài Gòn

GVHD: ThS. Phạm Thi Vương

Huỳnh Thanh Tiến – 3123410374
Phan Hữu Kha – 3123560038
Võ Thị Như Ý – 3123410439
Trần Hà Thanh Thùy – 3123560087

TP. HỒ CHÍ MINH, THÁNG 10/2026

MỤC LỤC

LỜI MỞ ĐẦU 6
1. Lý do chọn đề tài 6
2. Mục tiêu báo cáo 6
3. Phạm vi nghiên cứu 7
4. Kế hoạch và tiến độ 12 tuần 7
5. Cấu trúc báo cáo 8
CHƯƠNG 1. GIỚI THIỆU VỀ LLAMAINDEX 9
1.1 Bối cảnh và lịch sử 9
1.1.1 Bài toán: LLM không biết dữ liệu riêng 9
1.1.2 LlamaIndex ra đời từ đâu 9
1.2 Kiến trúc và nguyên lý hoạt động 10
1.2.1 Retrieval-Augmented Generation (RAG) 10
1.2.2 Embedding và độ tương đồng vector 11
1.2.3 Cắt đoạn văn bản (chunking) 11
1.2.4 Chỉ mục vector và truy hồi top-k 12
1.2.5 Tổng hợp câu trả lời (Response Synthesis) 12
1.2.6 Xếp hạng lại (reranking) 13
1.2.7 Các thành phần chính và luồng xử lý 13
1.2.8 Hệ sinh thái 14
1.3 So sánh và đánh giá 14
1.3.1 So với tự viết RAG 14
1.3.2 So với LangChain 15
1.3.3 Điểm ma sát quan sát được khi dùng thật 16
1.3.4 Khi nào KHÔNG nên dùng LlamaIndex 17
CHƯƠNG 2. VỊ THẾ CÔNG NGHỆ VÀ PHẠM VI ỨNG DỤNG 17
2.1 Các trường hợp ứng dụng tiêu biểu 17
2.2 Tình hình sử dụng 18
2.3 Xu hướng và giới hạn 18
2.4 Kết luận phạm vi 19
CHƯƠNG 3. THIẾT LẬP MÔI TRƯỜNG VÀ KIẾN TRÚC DỰ ÁN 20
3.1 Công cụ cần thiết 20
3.1.1 Thư viện Python 20
3.1.2 Phụ thuộc hệ thống 21
3.1.3 LLM chạy qua Ollama 21

3.1.4 Phần cứng và bộ nhớ GPU **21**
3.2 Cài đặt và Hello World **22**
3.2.1 Các bước cài đặt **22**
3.2.2 Hello World **22**
3.3 Minh chứng kiến trúc **23**
3.3.1 Cấu trúc thư mục dự án **23**
3.3.2 Cấu hình tập trung và hai profile **27**
3.3.3 Các script hỗ trợ **28**
3.4 Kiểm chứng môi trường **30**
CHƯƠNG 4. BẢN CHẤT CÔNG NGHỆ: BẮT BUỘC LỖI **31**
4.1 Nạp dữ liệu (Ingestion) **31**
4.1.1 Vì sao cần OCR **31**
4.1.2 Cơ chế **32**
4.1.3 Kết quả chạy thật **33**
4.1.4 Ý nghĩa trong đồ án **34**
4.1.5 Mở rộng dữ liệu đợt 2–3 34
4.2 Chia Node/chunk **36**
4.2.1 Cơ chế **36**
4.2.2 Chunk size và overlap ở mức ứng dụng **36**
4.3 Embedding và lập chỉ mục **38**
4.3.1 Mô hình embedding tiếng Việt được chọn **38**
4.3.2 Cơ chế **38**
4.3.3 Kết quả chạy thật **39**
4.4 Truy hồi (Retriever) **39**
4.4.1 Cơ chế **39**
4.4.2 Kết quả chạy thật: top-5 kèm điểm **39**
4.5 Tổng hợp câu trả lời (Response synthesis) **41**
4.5.1 Cơ chế **41**
4.5.2 Sửa lỗi nhầm thang điểm hệ 10 và hệ 4 **42**
4.5.3 Hạn chế của cấu hình này **43**
4.6 Đánh giá thực nghiệm **43**
4.6.1 Độ đo **43**
4.6.2 Kết quả **44**
4.6.3 Câu hỏi tự kiểm **46**
4.6.4 Tập kiểm chứng chống quá khớp 46
CHƯƠNG 5. CUSTOM NODE PARSER THEO CẤU TRÚC VĂN BẢN PHÁP

QUY 48
5.1 Vì sao chọn 48
5.2 Cơ chế 48
5.2.1 Mã nguồn 49
5.3 Đã làm gì 49
5.4 Số liệu 49
5.5 Kiểm thử 50
5.6 Hạn chế đã biết 50
5.7 Dùng ở đâu trong đồ án 50
CHƯƠNG 6. PERSISTENT INDEX VÀ QUẢN LÝ CẤU HÌNH 51
6.1 Vì sao chọn 51
6.2 Cơ chế 51
6.2.1 Fingerprint: chống dùng lại chỉ mục cũ 51
6.3 Số liệu 52
6.4 Dùng ở đâu trong đồ án 52
CHƯƠNG 7. TRÍCH DẪN NGUỒN VÀ TỪ CHỐI CÂU HỎI NGOÀI CORPUS 53
7.1 Vì sao chọn 53
7.2 Cơ chế 53
7.3 Đã làm gì 53
7.4 Số liệu 53
7.5 Dùng ở đâu trong đồ án 54
CHƯƠNG 8. KỸ THUẬT NÂNG CAO ĐÃ CHỌN: HYBRID SEARCH VÀ XẾP HẠNG LẠI 55
8.1 Vì sao chọn 55
8.2 Cơ chế 55
8.3 Số liệu 57
8.4 Dùng ở đâu trong đồ án 58
CHƯƠNG 9. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG 60
9.1 Mô tả bài toán 60
9.2 User story 60
9.3 Thiết kế 60
9.3.1 Sơ đồ pipeline 60
9.3.2 Mô hình dữ liệu 61
9.3.3 Luồng truy vấn 61
CHƯƠNG 10. TRIỂN KHAI, TÍCH HỢP VÀ MINH CHỨNG KẾT QUẢ 63
10.1 Tích hợp hệ thống 63

10.2 Kết quả đạt được **63**
10.2.1 Truy hồi **63**
10.2.2 Câu trả lời **63**
10.2.3 So sánh trước – sau các tối ưu **70**
10.2.4 Hướng dẫn chạy lại **70**
10.3 Bảng phủ kỹ thuật **70**
CHƯƠNG 11. ĐÁNH GIÁ VÀ NHÌN LẠI **72**
11.1 So sánh với mục tiêu ban đầu **72**
11.2 Hạn chế **73**
11.3 Phân tích lỗi **74**
11.3.1 Lỗi 1: LLM nhầm thang điểm hệ 10 với hệ 4 **74**
11.3.2 Lỗi 2: CUDA out of memory khi nạp embedding cùng reranker **74**
11.4 AI Disclosure **74**
CHƯƠNG 12. HƯỚNG PHÁT TRIỂN **75**
12.1 Cải tiến sản phẩm **75**
12.2 Định hướng bản thân **76**
12.3 Kết luận **76**
TÀI LIỆU THAM KHẢO **77**
PHỤ LỤC A. NHẬT KÝ CÔNG VIỆC HÀNG TUẦN **78**
A.1 Tuần 5 (05–09/10/2026) **79**
A.1.1 Công việc đã làm **79**
A.1.2 Kết quả và khó khăn **79**
A.1.3 Việc chuyển sang tuần 6 **80**
A.2 Tuần 6 (12/10–16/10/2026) 80
A.2.1 Công việc đã làm 80
A.2.2 Kết quả và khó khăn 81
A.2.3 Việc chuyển sang tuần 7 81
PHỤ LỤC B. BẢNG TỰ KIỂM NHANH BA TẦNG **82**
PHỤ LỤC C. THỰC NGHIỆM HỌC TẬP **82**

LỜI MỞ ĐẦU

1. Lý do chọn đề tài

Quy chế đào tạo là văn bản mà sinh viên tra cứu thường xuyên trong suốt quá trình học: thời gian tối đa để hoàn thành khóa học, cách tính điểm học phần, điều kiện xét tốt nghiệp, thủ tục nghỉ học tạm thời hay chuyển ngành. Tại Trường Đại học Sài Gòn (SGU), các quy định này nằm trong nhiều văn bản dài, phần lớn là PDF bản scan, và có văn bản sửa đổi chỉ áp dụng cho một số khóa. Tìm theo từ khóa chỉ khớp khi người hỏi dùng đúng thuật ngữ của văn bản ("cấm thi" không tìm được "không đủ điều kiện dự thi"); hỏi thẳng một mô hình ngôn ngữ lớn (LLM) thì câu trả lời trôi chảy nhưng không có căn cứ và có thể bịa.

LlamaIndex được chọn vì nó chuyên cho bài toán hỏi đáp trên dữ liệu riêng và có sẵn các khâu nạp dữ liệu, cắt Node, lập chỉ mục, truy hồi, tổng hợp câu trả lời [1]; hệ sinh thái của nó tích hợp sẵn Ollama (chạy LLM cục bộ, dữ liệu không ra dịch vụ bên ngoài) và mô hình embedding tiếng Việt trên HuggingFace.

Định hướng nghề nghiệp của từng thành viên: *(Chưa làm — bổ sung ở bản báo cáo cuối)*.

2. Mục tiêu báo cáo

Mục tiêu kiến thức (cam kết). Sáu mục lõi Tầng 1A:

1. Ingestion: Document và metadata.
2. Node/chunk, giải thích chunk size và overlap.
3. Embedding và vector index.
4. Retriever top-*k* / similarity.
5. Response synthesis có truy vết nguồn.
6. Đánh giá thực nghiệm ở khâu truy hồi.

Mục tiêu kỹ năng (kế hoạch). Tầng 1B: custom node parser theo Điều, persistent index có fingerprint, trích dẫn nguồn, từ chối câu hỏi ngoài corpus, nhiều nguồn (20 văn bản). Tầng 3: hybrid BM25 + vector, xếp hạng lại bằng cross-encoder, đánh giá có khoảng tin cậy và kiểm định thống kê, model server cho embedding/reranker.

3. Phạm vi nghiên cứu

- **Corpus**: 20 văn bản SGU đang áp dụng, chia năm nhóm: quy chế đào tạo đại học (Quy chế 2021, QĐ 3183/2025 sửa đổi Quy chế, QĐ 810, QĐ 2375/2022, quy định khối lượng đào tạo, quy định 150 tín chỉ kỹ sư), quy đổi chứng chỉ tiếng Anh theo khóa (TB 2366, TB 2451, QyĐ 2626, QyĐ 2101, TB 2567), thi kết thúc học phần (QĐ 898/2025, QĐ 3260 về hoãn thi, Quy định 3248/2018, HD 1181 về phúc khảo), chuẩn đầu ra ngoại ngữ (TB 536/2020) và học vụ khác (cố vấn học tập, cấp bảng điểm, cấp bản sao, tiết học và phòng học). Tổng cộng 118 Node.
- **Mô hình**: embedding AITeamVN/Vietnamese_Embedding chạy local (GPU 4 GB ở fp16 hoặc CPU); LLM qwen3:8b qua Ollama trên máy GPU thuê hoặc Google Colab.
- **Không làm**: tư vấn pháp lý, agent, fine-tune LLM. Giao diện web chỉ ở mức chạy thử (Gradio).

Thay đổi phạm vi so với kế hoạch ban đầu. Kế hoạch ban đầu là một văn bản (Quy chế 2021, 17 trang scan) với LLM qwen3:4b chạy hoàn toàn trên máy cá nhân. Corpus được mở rộng dần: 6 văn bản ở tuần 4 để trả lời được nội dung sửa đổi, rồi 20 văn bản ở tuần 6 để trả lời các câu sinh viên hỏi nhiều về quy đổi chứng chỉ tiếng Anh, thi và phúc khảo. LLM chuyển sang chạy từ xa vì GPU 4 GB không chứa nổi qwen3:8b cùng embedding. Lý do chi tiết ở Mục 11.1.

4. Kế hoạch và tiến độ 12 tuần

Bảng 0.1: Kế hoạch 12 tuần và trạng thái tại tuần 6 (12/10–16/10/2026); việc của các tuần sau đã làm trước lịch được ghi rõ ở cột việc cụ thể. Người thực hiện từng tuần ghi ở Phụ lục A.

Tuần	Nội dung chính	Việc cụ thể của nhóm	Trạng thái
1	Lập nhóm, chọn công nghệ	Đăng ký LlamaIndex, hồ sơ RAG	Xong
2	Môi trường, Hello World, repo	venv, Ollama, hello_world.py, check_model_memory.py	Xong
3–4	Bắt buộc lõi, nháp Ch. 3–4	OCR, Node theo Điều, index; bộ câu hỏi 01/02	Xong; thay bằng Markdown hai tầng
5	Chốt bài toán, thiết kế luồng	6 văn bản, sơ đồ pipeline; Colab; chạy lại 56 câu	Xong
6–7	Chọn thêm, thực nghiệm	Retriever, query engine, Recall@k; nạp đợt 2–3 (20 văn bản); tập kiểm	Đang làm

Tuần	Nội dung chính	Việc cụ thể của nhóm	Trạng thái
		chứng 60 câu, hai người gán nhãn; thí nghiệm chunking chưa chạy	
8–9	Tích hợp, Tầng 3	Trích nguồn, rerank, hybrid, lọc phiên bản (làm trước lịch); từ chối ngoài corpus	Một phần
10	Kiểm thử, đo trước–sau	54 kiểm thử + CI (làm trước lịch); chấm câu trả lời tự động	Một phần
11	Hands-on lab, chấm chéo	Viết lab	Chưa
12	Hoàn thiện, niêm phong	Báo cáo, slide, video; nộp ≥ 24 giờ trước bảo vệ	Chưa

5. Cấu trúc báo cáo

Chương 1–2 giới thiệu LlamaIndex, kiến trúc, so sánh dựa trên trải nghiệm của nhóm và phạm vi nên/không nên dùng. Chương 3–8 trình bày môi trường, sáu mục lõi Tầng 1A với mã nguồn và kết quả chạy thật, rồi các kỹ thuật chọn thêm (node parser, persistent index, trích dẫn/từ chối) và nâng cao (hybrid, rerank). Chương 9–10 là phân tích thiết kế, triển khai, minh chứng và Bảng phủ kỹ thuật. Chương 11–12 đánh giá lại so với mục tiêu, phân tích lỗi, AI Disclosure và hướng phát triển. Cuối báo cáo là tài liệu tham khảo và các phụ lục: nhật ký tuần, bảng tự kiểm ba tầng và thực nghiệm học tập.

Mọi số liệu đều lấy từ việc chạy thật các script trong repository; phần chưa làm được đánh dấu bằng khung “Chưa triển khai” hoặc ghi chú “Chưa làm” kèm thời điểm dự kiến bổ sung.

CHƯƠNG 1. GIỚI THIỆU VỀ LLAMAINDEX

1.1 Bối cảnh và lịch sử

1.1.1 Bài toán: LLM không biết dữ liệu riêng

Mô hình ngôn ngữ lớn (Large Language Model, LLM) là mạng nơ-ron được huấn luyện trên lượng văn bản rất lớn để dự đoán token tiếp theo. Nhờ đó LLM có khả năng hiểu câu hỏi, diễn đạt lại, tóm tắt và suy luận đơn giản. Tuy nhiên, khi dùng LLM để trả lời về một văn bản nội bộ như quy chế của SGU, có bốn hạn chế cần xử lý:

1. **Thiếu tri thức riêng.** Quy chế của một trường cụ thể, nhất là bản ban hành gần đây, hầu như không nằm trong dữ liệu huấn luyện, hoặc chỉ xuất hiện rất mờ nhạt.
2. **Tri thức bị đóng băng.** Kiến thức của mô hình dừng ở thời điểm huấn luyện; khi quy chế được sửa đổi, mô hình không tự cập nhật.
3. **Ảo giác (hallucination).** Mô hình có xu hướng sinh ra câu trả lời nghe hợp lý nhưng sai, ví dụ bịa số tín chỉ hoặc số Điều.
4. **Khó kiểm chứng.** Câu trả lời không kèm nguồn nên người dùng không biết đối chiếu ở đâu.

Có thể khắc phục bằng cách tinh chỉnh (fine-tuning) mô hình trên văn bản quy chế, nhưng cách này tốn kém, phải huấn luyện lại mỗi khi văn bản đổi, và vẫn không loại bỏ hoàn toàn ảo giác. RAG đi theo hướng khác: giữ nguyên mô hình, đưa tri thức cần thiết vào *ngữ cảnh* (prompt) tại thời điểm hỏi.

1.1.2 LlamaIndex ra đời từ đâu

LlamaIndex do Jerry Liu khởi xướng, ra mắt cuối năm 2022 dưới tên *GPT Index*, và đổi tên thành LlamaIndex trong năm 2023 [2]. Dự án hiện do công ty LlamaIndex Inc. phát triển: thư viện mã nguồn mở (gói `llama-index-core` và các gói tích hợp) đi kèm các dịch vụ thương mại như LlamaCloud và LlamaParse.

Điểm xuất phát của dự án cũng chính là pain-point nêu trên: LLM không biết dữ liệu riêng, cửa sổ ngữ cảnh có hạn nên không thể nhét cả kho tài liệu vào prompt, và khi thiếu căn cứ thì mô hình bịa. LlamaIndex giải quyết bằng một *lớp dữ liệu* đứng giữa tài liệu và

LLM: nạp tài liệu, cắt thành đơn vị nhỏ, lập chỉ mục, rồi khi có câu hỏi thì truy hồi đúng phần cần thiết đưa vào prompt.

1.2 Kiến trúc và nguyên lý hoạt động

Gọi đúng loại. LlamaIndex là một **thư viện/framework dữ liệu cho ứng dụng LLM**. Nó không phải là mô hình ngôn ngữ, và cũng không phải "RAG": RAG là một *kiến trúc* (truy hồi rồi sinh), còn LlamaIndex là công cụ cài đặt kiến trúc đó (và cả các kiến trúc khác như agent).

1.2.1 Retrieval-Augmented Generation (RAG)

1.2.1.1 Ý tưởng

RAG [3], [4] tách bài toán thành hai bước: (1) *truy hồi* — từ kho tài liệu, tìm ra một số ít đoạn liên quan nhất tới câu hỏi; (2) *sinh* — đưa câu hỏi cùng các đoạn đó cho LLM và yêu cầu trả lời dựa trên chúng. Nếu ký hiệu q là câu hỏi, D là tập các đoạn văn bản trong kho, thì:

$$C = Retrieve(q, D, k), \qquad a = LLM(prompt(q, C)),$$

trong đó $C \subset D$ là k đoạn được chọn và a là câu trả lời. Vì C lấy trực tiếp từ quy chế nên câu trả lời có căn cứ, và hệ thống có thể trích dẫn nguồn (số Điều, số trang) cho người dùng kiểm tra.

1.2.1.2 Hai pha của một hệ RAG

Một hệ RAG hoạt động theo hai pha có tần suất chạy khác nhau:

- **Pha chuẩn bị (offline), chạy khi tài liệu thay đổi:** nạp tài liệu → cắt thành các đoạn → biến mỗi đoạn thành vector embedding → lưu vào chỉ mục.
- **Pha truy vấn (online), chạy mỗi câu hỏi:** embedding câu hỏi → tìm k đoạn gần nhất → ghép prompt → LLM sinh câu trả lời.

Chất lượng của hệ thống bị giới hạn bởi *đoạn văn tìm được*: nếu bước truy hồi bỏ sót Điều đúng thì LLM dù tốt đến đâu cũng không thể trả lời đúng. Vì vậy phần lớn công sức của đồ án tập trung vào các bước chuẩn bị dữ liệu (nạp, cắt đoạn) và truy hồi.

1.2.2 Embedding và độ tương đồng vector

1.2.2.1 Vector embedding

Embedding là phép ánh xạ một đoạn văn bản thành một vector số thực $v \in R^d$ sao cho các đoạn có nghĩa gần nhau nằm gần nhau trong không gian vector. Với mô hình dạng bi-encoder [5], câu hỏi và đoạn văn được mã hóa *độc lập* bởi cùng một bộ mã hóa; nhờ vậy vector của toàn bộ kho tài liệu có thể tính trước và lưu lại, khi có câu hỏi chỉ cần mã hóa riêng câu hỏi. Chất lượng của mô hình embedding là yếu tố ảnh hưởng mạnh nhất tới kết quả truy hồi, và được xếp hạng công khai trên các benchmark như MTEB [6].

1.2.2.2 Độ đo cosine

Độ tương đồng giữa vector câu hỏi q và vector đoạn văn d thường được đo bằng cosine:

$$sim(q,d) = \frac{q \cdot d}{\|q\| \, \|d\|} = \frac{\sum_{i=1}^{d} q_i d_i}{\sqrt{\sum_{i=1}^{d} q_i^2} \sqrt{\sum_{i=1}^{d} d_i^2}}.$$

Giá trị càng gần 1 thì hai đoạn càng gần nhau về ngữ nghĩa. Cách khác là so khớp theo từ khóa xác suất kiểu BM25 [7]: mạnh ở token hiếm và ký hiệu ("B+", "150 tín chỉ") nhưng không hiểu diễn đạt khác nghĩa; vì hai cách sai ở hai chỗ khác nhau, chúng thường được trộn thành *hybrid search*. Khác với tìm theo từ khóa, so khớp vector cho phép "cấm thi" tìm được đoạn viết "không đủ điều kiện dự thi" vì hai cụm có embedding gần nhau.

1.2.3 Cắt đoạn văn bản (chunking)

1.2.3.1 Vì sao phải cắt

Có hai lý do phải cắt tài liệu thành các đoạn nhỏ. Thứ nhất, cửa sổ ngữ cảnh của LLM có hạn, và càng nhét nhiều văn bản không liên quan thì câu trả lời càng dễ lệch — mô hình còn có xu hướng bỏ qua thông tin nằm giữa ngữ cảnh dài [8]. Thứ hai, một vector embedding cho cả tài liệu sẽ "pha loãng" mọi ý; mỗi đoạn nên xoay quanh một chủ đề để vector của nó đại diện đúng.

1.2.3.2 Các chiến lược cắt

Bảng 1.1: So sánh ba chiến lược cắt đoạn.

Chiến lược	**Ưu điểm**	**Nhược điểm**
Cắt theo độ dài cố định	Đơn giản, kích thước đều	Có thể cắt ngang câu hoặc ngang điều khoản
Cắt theo câu (sentence splitter)	Không gãy câu, có chồng lấp (overlap)	Vẫn không biết ranh giới điều khoản
Cắt theo cấu trúc văn bản	Mỗi đoạn là một đơn vị nghĩa trọn vẹn	Phụ thuộc định dạng văn bản, cần quy tắc riêng

Ứng dụng cụ thể của các chiến lược này vào quy chế SGU nằm ở Mục 4.2 và Chương 5.

1.2.4 Chỉ mục vector và truy hồi top-*k*

Sau khi có vector cho mọi đoạn, hệ thống cần lưu chúng theo cách tìm kiếm nhanh. *Chỉ mục vector* (vector index) lưu các cặp (vector, đoạn văn, metadata). Khi có câu hỏi, bộ truy hồi (retriever) tính độ tương đồng giữa vector câu hỏi và các vector trong chỉ mục rồi trả về k đoạn có điểm cao nhất (top-k) [9]. Với kho nhỏ như một quy chế (vài chục đến vài trăm đoạn), việc so sánh vét cạn với mọi vector là đủ nhanh; các cấu trúc tìm kiếm xấp xỉ chỉ cần khi kho rất lớn.

1.2.5 Tổng hợp câu trả lời (Response Synthesis)

Sau truy hồi, các đoạn tìm được phải được đưa cho LLM để sinh câu trả lời. LlamaIndex gọi bước này là *response synthesis* và cung cấp nhiều chế độ (response_mode). Các chế độ thường gặp:

- refine: gọi LLM lần lượt từng đoạn, mỗi lần tinh chỉnh câu trả lời cũ bằng đoạn mới; chính xác nhưng tốn nhiều lượt gọi.
- compact: gộp nhiều đoạn vào cùng một prompt sao cho vừa cửa sổ ngữ cảnh rồi mới gọi LLM, giảm số lượt gọi so với refine.
- tree_summarize: tóm tắt theo cấu trúc cây, phù hợp câu hỏi tổng hợp trên nhiều đoạn.

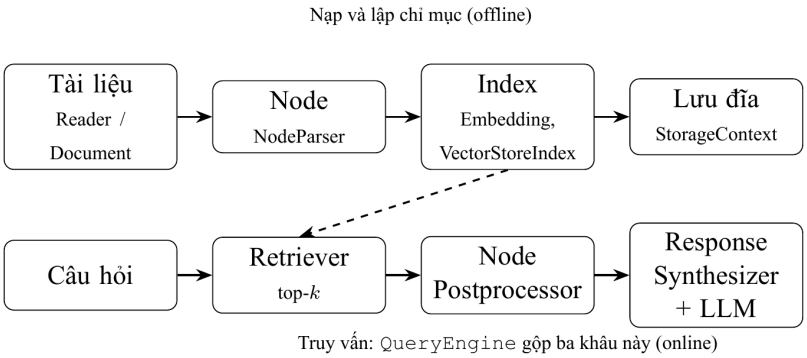
- simple_summarize: cắt bớt để nhét mọi đoạn vào một lần gọi.

1.2.6 Xếp hạng lại (reranking)

Mô hình embedding dạng bi-encoder mã hóa câu hỏi và đoạn văn *riêng rẽ*, nên nhanh nhưng chỉ so khớp ở mức "ý chính". Mô hình *cross-encoder* nhận *cặp* (câu hỏi, đoạn văn) làm một đầu vào duy nhất và chấm một điểm liên quan; nhờ xem cả hai cùng lúc, nó thường đánh giá chính xác hơn, nhưng phải chạy lại cho từng cặp nên chậm hơn nhiều và không thể tính trước. Vì vậy cách dùng phổ biến là hai tầng: bi-encoder lấy nhanh top-k ứng viên, rồi cross-encoder chấm lại và chỉ giữ top-n ($n < k$) đưa vào prompt.

1.2.7 Các thành phần chính và luồng xử lý

LlamaIndex [1] mô hình hóa chuỗi RAG bằng một số đối tượng có tên rõ ràng. Hình 1.1 vẽ lại luồng ba pha nạp — lập chỉ mục — truy vấn; bảng dưới chỉ ra từng đối tượng nằm ở đâu trong đồ án.



Hình 1.1: Luồng xử lý của LlamaIndex (vẽ lại theo tài liệu chính hãng).

Bảng 1.2: Các đối tượng chính của LlamaIndex và vị trí trong đồ án.

Khái niệm	**Vai trò**	**Trong đồ án**
Document	Một nguồn dữ liệu thô kèm metadata	Chỉ dùng trong hello_world.py; dữ liệu thật được dựng thẳng thành TextNode
Node / TextNode	Đơn vị được lập chỉ mục, sinh từ Document hoặc dựng trực tiếp	processed_loader.py (mỗi Điều / mục sửa đổi một Node)
NodeParser	Cắt Document thành Node (SentenceSplitter, …)	SentenceSplitter cho Điều dài hơn 1 024 từ

Khái niệm	Vai trò	Trong đồ án
Embedding	Biến văn bản thành vector	HuggingFaceEmbedding hoặc RemoteEmbedding
VectorStoreIndex	Lưu Node kèm vector để truy hồi	index_builder.py
StorageContext	Ghi / nạp chỉ mục xuống đĩa	storage/<profile>/<corpus>
Retriever	Lấy các Node liên quan tới câu hỏi	retriever.py (cosine top-k, tuỳ chọn trộn BM25)
NodePostprocessor	Lọc, xếp hạng lại Node sau truy hồi	retriever.py (ngưỡng, reranker), phien_ban.py (lọc phiên bản)
Response synthesizer	Ghép Node vào prompt và gọi LLM	query_engine.py (chế độ compact)
QueryEngine	Gộp retriever, postprocessor và synthesizer	query_engine.py (RetrieverQueryEngine)
Settings	Cấu hình toàn cục (embed_model, llm)	Chỉ gán ở index_builder và query_engine

Điểm mạnh của thiết kế này là mỗi khâu độc lập: có thể đổi mô hình embedding, đổi cách cắt đoạn hoặc đổi chế độ tổng hợp mà không phải viết lại toàn bộ hệ thống.

1.2.8 Hệ sinh thái

Ngoài llama-index-core, LlamaIndex chia các tích hợp thành gói riêng: reader trên LlamaHub (PDF, web, CSDL…), vector store (Chroma, Qdrant, …), LLM (Ollama, OpenAI, …) và embedding (HuggingFace, …). Đồ án dùng hai gói tích hợp: llama-index-llms-ollama và llama-index-embeddings-huggingface. Cách chia gói này có mặt trái: BM25Retriever không nằm trong core, nên đồ án tự viết (Mục 8.2).

1.3 So sánh và đánh giá

1.3.1 So với tự viết RAG

Đồ án có hai chỗ vừa dùng LlamaIndex vừa tự viết cùng chức năng, nên so sánh được bằng trải nghiệm thật:

- **Đánh giá truy hồi.** eval/metrics.py tự viết Recall@k, MRR, nDCG, bootstrap và McNemar; RetrieverEvaluator của LlamaIndex cho cùng mrr đến ba chữ số thập phân

(Mục 4.6). Thư viện đủ cho hit rate và MRR, nhưng không có khoảng tin cậy và kiểm định cặp — phần đó vẫn phải tự viết.

- **BM25.** Tự viết khoảng 100 dòng (src/bm25.py) trên BaseRetriever; nhờ kế thừa interface của LlamaIndex, nó cắm thẳng vào QueryFusionRetriever mà không phải sửa phần còn lại.

Những gì LlamaIndex tiết kiệm được so với tự viết bằng sentence-transformers + numpy: lưu/nạp chỉ mục (StorageContext), chọn trường metadata nào vào embedding và prompt (các danh sách excluded_…_metadata_keys), trộn nhiều retriever, chuỗi postprocessor, response synthesizer có streaming và chế độ compact. Những gì nó *không* tiết kiệm được nằm ở bảng ma sát dưới đây.

1.3.2 So với LangChain

Để so sánh bằng thực nghiệm, nhóm dựng lại riêng phần truy hồi vector bằng LangChain (eval/so_sanh_langchain.py, langchain-core 1.6.12, InMemoryVectorStore) trên cùng 118 Node, cùng mô hình AITeamVN/Vietnamese_Embedding (fp16, chuẩn hóa L2), cùng top-10 và cùng luật chấm. Hai cách cài cho kết quả trùng khớp: R@1 82,4% trên 91 câu dev, và không câu nào có hạng câu trả lời đúng khác nhau. Khác biệt giữa hai framework vì vậy nằm ở mức trừu tượng (Bảng 1.4). Phần sinh câu trả lời chưa được dựng lại bằng LangChain.

Bảng 1.4: Cùng phần truy hồi cài bằng LlamaIndex và LangChain, đo ngày 17/10/2026.

Tiêu chí	**LlamaIndex**	**LangChain**
R@1 / R@3 trên 91 câu dev	82,4% / 95,6%	Giống hệt; 0/91 câu khác hạng
Đơn vị dữ liệu	TextNode: nội dung, metadata, quan hệ, id_	Document: nội dung và metadata dạng dict
Chọn metadata đưa vào embedding và vào LLM	excluded_embed_metadata_keys , excluded_llm_metadata_keys trên từng Node	Không có cơ chế tương đương; phải tự ghép chuỗi đem embed vào page_content
Mô hình embedding HuggingFace	HuggingFaceEmbedding có sẵn	langchain-core không có; tự viết lớp Embeddings (9 dòng) hoặc cài thêm langchain-huggingface

Lưu chỉ mục	StorageContext.persist() và load_index_from_storage(), kèm docstore	InMemoryVectorStore.dump() và load(); phát hiện corpus đổi phải tự làm
Số dòng phần truy hồi	get_retriever(): 26 dòng, gồm cả nhánh hybrid BM25	9 dòng, cộng 9 dòng lớp Embeddings; không có hybrid

1.3.3 Điểm ma sát quan sát được khi dùng thật

Phần này ghi lại những chỗ thư viện không làm đúng như kỳ vọng ban đầu. Đây là loại thông tin mà tài liệu chính thức không nêu, và là một trong những kết quả của việc *dùng thật* thư viện.

Bảng 1.3: Các điểm ma sát với LlamaIndex gặp trong quá trình làm đồ án.

Hiện tượng	**Nguyên nhân**	**Cách xử lý**
Không nạp được embedding và reranker cùng lúc trên GPU 4 GB	SentenceTransformerRerank không có tham số dtype, luôn nạp fp32	Nạp reranker trên CPU, ép _model.half() rồi mới chuyển lên GPU (dùng API nội bộ)
Vector bị nhiễu bởi các trường metadata rỗng	Mặc định metadata_template là {key}: {value} cho *mọi* khóa, kể cả rỗng	Lọc khóa rỗng và chỉ để một trường (nguon_trich) được embedding
Không gắn được nhãn tiếng Việt cho từng khóa metadata	metadata_template chỉ có hai chỗ điền {key}, {value}, không tùy biến theo khóa	Gom nhãn vào một trường nguon_trich tự sinh
Bật hybrid là lỗi ImportError: llama-index-llms-openai	QueryFusionRetriever làm llm if llm else Settings.llm, mà Settings.llm mặc định trỏ tới OpenAI	Truyền thẳng đối tượng Ollama vào retriever (với num_queries=1 nó không bao giờ được gọi)
BM25Retriever không có trong llama-index-core	Thư viện tách phần này ra package riêng (llama-index-retrievers-bm25)	Tự viết src/bm25.py trên BaseRetriever (xem Mục 8.2)
Không lọc được theo toán tử "khác" trên metadata	SimpleVectorStore mặc định chỉ là JSON trong RAM, hỗ trợ lọc hạn chế	Chấp nhận ở quy mô hiện tại; ghi nhận việc chuyển sang Chroma/Qdrant khi corpus lớn
Đổi mô hình embedding là phải build lại toàn bộ chỉ mục	Định dạng lưu default__vector_store.json gắn với số chiều vector	Tách thư mục chỉ mục theo profile và thêm fingerprint theo model

Hiện tượng	Nguyên nhân	Cách xử lý
Cấu hình toàn cục gây tác dụng phụ	Settings là trạng thái global (embed_model, llm)	Chỉ gán Settings ở đúng hai chỗ (index_builder, query_engine)
Reranker chấm sai Node dài (bảng, Điều dài)	SentenceTransformerRerank tạo CrossEncoder với max_length = 512 cố định, không có tham số để đổi	Đặt lại max_seq_length = 2 048 sau khi nạp (dat_do_dai_toi_da() trong src/retriever.py)

Một quan sát tổng quát: LlamaIndex tách bạch rất tốt ở mức *interface* (retriever, node postprocessor, response synthesizer đều thay được độc lập), nhưng ở mức *mặc định* thì thiên về "chạy được ngay" hơn là "đúng cho tiếng Việt và cho văn bản pháp quy". Vì vậy phần lớn công sức của đồ án nằm ở việc thay các mặc định đó, chứ không ở việc ghép các thành phần lại.

1.3.4 Khi nào KHÔNG nên dùng LlamaIndex

1. **Câu hỏi cần tính toán hoặc tra bảng có cấu trúc.** Quy đổi điểm hệ 10 sang hệ 4 là phép tra bảng xác định. Đồ án ghi nhận LLM tự "quy đổi" 8,4 thành 3,4 và nhầm bảng hệ 10 với bảng hệ 4 (Mục 4.5.2) dù truy hồi đúng. Loại câu này nên giải bằng một hàm Python hoặc truy vấn SQL trên bảng quy đổi, không bằng truy hồi văn bản.
2. **Bài toán phân loại / dự đoán trên dữ liệu bảng.** Đó là việc của scikit-learn hay các thư viện học máy; LlamaIndex không có gì để đóng góp.
3. **Tài liệu ngắn, ít thay đổi, vừa cửa sổ ngữ cảnh.** Hai quy định một trang trong corpus nếu đứng riêng thì gọi thẳng LLM với toàn văn là đủ; dựng chỉ mục chỉ thêm độ phức tạp.

CHƯƠNG 2. VỊ THẾ CÔNG NGHỆ VÀ PHẠM VI ỨNG DỤNG

2.1 Các trường hợp ứng dụng tiêu biểu

- **Hỏi đáp trên tài liệu nội bộ**: quy chế, sổ tay, tài liệu kỹ thuật — đúng dạng bài toán của đồ án.
- **Trợ lý tra cứu chính sách**: văn bản có sửa đổi, có hiệu lực theo thời gian, cần trích dẫn nguồn để người dùng tự kiểm tra.

- **Tìm kiếm tri thức doanh nghiệp**: gom nhiều nguồn (PDF, wiki, CSDL) vào một chỉ mục và hỏi bằng ngôn ngữ tự nhiên.

2.2 Tình hình sử dụng

Số liệu truy cập ngày 17/10/2026 từ GitHub REST API và pypistats.org (Bảng 2.1). LlamaIndex có khoảng 52,5 nghìn sao và khoảng 2 000 người đóng góp trên GitHub; gói lõi llama-index-core được tải khoảng 6,6 triệu lượt mỗi tháng trên PyPI. LangChain có số liệu lớn hơn nhiều (khoảng 148 nghìn sao, 145 triệu lượt tải langchain-core mỗi tháng, một phần vì nhiều gói khác kéo nó theo làm phụ thuộc), còn Haystack nhỏ hơn (khoảng 26,7 nghìn sao, 0,57 triệu lượt tải haystack-ai mỗi tháng). Trong ba framework này, LlamaIndex đứng thứ hai về mức độ sử dụng, với cộng đồng đủ lớn để tài liệu và các gói tích hợp được cập nhật thường xuyên. Nhóm chưa thu thập số tin tuyển dụng.

Bảng 2.1: Mức độ sử dụng LlamaIndex, LangChain và Haystack, truy cập ngày 17/10/2026. Người đóng góp tính theo GitHub API, kể cả tài khoản ẩn danh.

Dự án (repo GitHub)	**Sao GitHub**	**Người đóng góp**	**Lượt tải PyPI 30 ngày (gói)**
LlamaIndex (run-llama/llama_index)	52 524	khoảng 2 007	6 574 473 (llama-index-core 0.14.25)
LangChain (langchain-ai/langchain)	147 936	khoảng 3 745	144 826 657 (langchain-core)
Haystack (deepset-ai/haystack)	26 747	khoảng 508	566 454 (haystack-ai)

2.3 Xu hướng và giới hạn

- **Xu hướng**: từ RAG một lượt sang *Workflows* (luồng nhiều bước điều khiển bằng sự kiện) và agent dùng công cụ; dịch vụ phân tích tài liệu (LlamaParse) cho PDF phức tạp.

- **Đối thủ**: LangChain/LangGraph (thiên về chuỗi và agent tổng quát), Haystack (thiên về pipeline tìm kiếm). LlamaIndex mạnh nhất ở tầng dữ liệu: nạp, cắt, lập chỉ mục, truy hồi.
- **Giới hạn quan sát được**: mặc định thiên về tiếng Anh và OpenAI (Settings.llm trỏ tới OpenAI); nhiều tích hợp tách gói nên phải tự ghim phiên bản (Mục 1.3.3).

2.4 Kết luận phạm vi

Bảng 2.2: Phạm vi nên và không nên dùng LlamaIndex, rút ra từ đồ án.

Nên dùng LlamaIndex	**Không nên dùng**
Hỏi đáp trên kho văn bản riêng, cần trích nguồn	Tính toán, tra bảng xác định, truy vấn có cấu trúc
Kho tài liệu thay đổi, cần lập chỉ mục lại	Phân loại / dự đoán trên dữ liệu bảng
Cần thử nhiều cách cắt, truy hồi, xếp hạng mà không viết lại hệ thống	Tài liệu ngắn vừa cửa sổ ngữ cảnh

CHƯƠNG 3. THIẾT LẬP MÔI TRƯỜNG VÀ KIẾN TRÚC DỰ ÁN

3.1 Công cụ cần thiết

3.1.1 Thư viện Python

File requirements.txt đã được **ghim đúng phiên bản** mà báo cáo dùng, để người cài sau nhận đúng các phiên bản đó. Bảng dưới ghi phiên bản đo được trong venv tại thời điểm chạy lại toàn bộ số liệu.

Bảng 3.1: Thư viện Python trong requirements.txt.

Gói	Phiên bản đo được	Mục đích
llama-index-core	0.14.24	Framework RAG
llama-index-embeddings-huggingface	0.8.0	Nạp mô hình embedding
llama-index-llms-ollama	0.11.0	Kết nối LLM qua Ollama
sentence-transformers	6.0.1	Chạy embedding/reranker
torch (cài kèm)	2.14.0 (CUDA 13.0)	Chạy mô hình trên GPU
sentencepiece, protobuf	0.2.2, không đo	Phụ thuộc của tokenizer
pytesseract	0.3.13	Gọi Tesseract từ Python
pdf2image	1.17.0	Render PDF thành ảnh
httpx	0.28.1	Gọi HTTP tới Ollama và model server
ollama	0.6.2	Client Python của Ollama (bắt lỗi trong app/app.py)
gradio	6.27.0	Giao diện web chạy thử (app/app.py)
python-dotenv	1.2.3	Đọc biến môi trường từ .env

3.1.2 Phụ thuộc hệ thống

- **Tesseract OCR** 5.5.3, đã xác nhận có gói ngôn ngữ vie.
- **poppler-utils** (pdftoppm 26.01.0), cần cho pdf2image.
- **Ollama** để chạy LLM (script pull_models.sh kiểm tra và báo lỗi nếu chưa cài).
- Python 3.14.6 trong venv.

3.1.3 LLM chạy qua Ollama

Để dữ liệu không đi qua dịch vụ AI bên thứ ba, LLM được chạy qua **Ollama**, một trình phục vụ mô hình mở với API HTTP (mặc định tại http://localhost:11434); ở profile máy chủ, Ollama chạy trên một máy GPU thuê riêng và địa chỉ được đọc từ biến OLLAMA_BASE_URL. Đồ án dùng họ mô hình Qwen3: qwen3:4b cho profile chạy cục bộ và qwen3:8b cho profile máy chủ. LlamaIndex kết nối tới Ollama [10] qua gói llama-index-llms-ollama; Qwen3 là dòng mô hình mở của Alibaba, bản 4B và 8B đủ nhỏ để chạy trên máy cá nhân [11]. Thời gian chờ mặc định trong cấu hình là 180 giây do mô hình chạy trên phần cứng cá nhân có thể trả lời chậm. Qwen3 có chế độ "suy nghĩ" (sinh chuỗi lập luận trước câu trả lời); đồ án tắt chế độ này (thinking=False) để câu trả lời ngắn và nhanh.

3.1.4 Phần cứng và bộ nhớ GPU

Máy phát triển dùng card rời NVIDIA RTX A2000 Laptop (4 GB VRAM, khoảng 3,7 GB khả dụng cho PyTorch). Kiểm tra bằng cách nạp mô hình qua đúng hàm của dự án và in thiết bị của các tham số cho thấy cả mô hình embedding lẫn reranker đều nằm trên cuda:0. LLM không chạy trên máy này: ở profile server, Ollama chạy trên máy GPU thuê từ dịch vụ ckey.vn, được quản lý bằng scripts/manager.py.

Trên card 4 GB, embedding và reranker chỉ cùng nằm được trên GPU khi nạp ở fp16: ở fp32, tiến trình chiếm khoảng 3,5 GB sau khi nạp embedding và báo CUDA out of memory khi nạp mô hình thứ hai. Ở fp16, embedding còn khoảng 1,1 GB và cả hai mô hình tổng khoảng 2,2 GB (src/index_builder.py và make_local_reranker() trong src/retriever.py; quá trình xử lý lỗi ở Mục 11.3.2). Hàm resolve_device() tự hạ cấp về CPU khi máy không có CUDA, nên cùng mã nguồn chạy được trên máy GPU, máy CPU và CI. Số liệu truy hồi ở Mục 4.6 và Chương 8 được đo ngày 17/10/2026 trên chính card này.

3.2 Cài đặt và Hello World

3.2.1 Các bước cài đặt

Trình tự đã dùng trong dự án:

1. Tạo và kích hoạt môi trường ảo Python (venv).
2. Cài các gói Python: pip install -r requirements.txt.
3. Cài Tesseract (kèm gói ngôn ngữ vie) và poppler-utils bằng trình quản lý gói của hệ điều hành.
4. Cài Ollama, chạy ollama serve, rồi tải mô hình bằng scripts/pull_models.sh [local|server].

3.2.2 Hello World

Script scripts/hello_world.py kiểm tra nhanh rằng LlamaIndex cài đúng: tạo một Document, dựng SummaryIndex và đếm số Document trong docstore. Script này không cần LLM hay mô hình embedding nên chạy được ngay sau khi cài thư viện.

Đoạn mã 3.1: Tạo Document và SummaryIndex với LlamaIndex (scripts/hello_world.py).

```
from llama_index.core import Document, SummaryIndex

doc = Document(text = "Điều 15. Sinh viên nghỉ học quá 20% số tiết sẽ không đủ điều kiện dự thi.")
print("Document tạo thành công!")
print("Nội dung:", doc.text)
print("ID tự sinh:", doc.doc_id)

index = SummaryIndex.from_documents([doc])
print("\nIndex build thành công!")
print("Số Document trong Index:", len(index.docstore.docs))
```

Kết quả chạy thật:

Đoạn mã 3.2: Kết quả chạy hello_world.py.

```
Document tạo thành công!
Nội dung: Điều 15. Sinh viên nghỉ học quá 20% số tiết sẽ không đủ điều kiện dự thi.
ID tự sinh: 3dc12976-3b4e-4711-8798-c7d0237d98a9

Index build thành công!
Số Document trong Index: 1
```

Ý nghĩa: doc_id được LlamaIndex tự sinh dạng UUID, và SummaryIndex lưu Document vào docstore. Câu ví dụ "Điều 15" trong script chỉ là văn bản minh họa tự viết, không phải nội dung trích từ quy chế.

3.3 Minh chứng kiến trúc

3.3.1 Cấu trúc thư mục dự án

Đoạn mã 3.3: Cấu trúc thư mục dự án.

```
.
|-- .github/
|   `-- workflows/
|       `-- ci.yml            # Cấu hình CI/CD tự động cho dự án
|-- .vscode/
|   `-- settings.json         # Cấu hình môi trường làm việc VS Code
|-- app/
|   |-- app.py                # Giao diện Gradio chạy thử demo RAG
|   `-- assets/               # Tài nguyên tĩnh cho ứng dụng giao diện
|-- data/
|   |-- bang_quy_doi/         # CSV bảng quy đổi chứng chỉ tiếng Anh và biên bản đối chiếu với PDF gốc
|   |-- raw/                  # Tầng 0: Dữ liệu thô (PDF, DOCX) chia theo 5 mảng công tác
|   |   |-- 01_quy_che_dao_tao/
|   |   |-- 02_cong_tac_sinh_vien/
|   |   |-- 03_chuong_trinh_dao_tao/
|   |   |-- 04_khoa_luan_thuc_tap/
|   |   `-- 05_bieu_mau_thu_tuc/
|   |-- ocr/                  # Tầng 1: Markdown thô sau OCR
|   |   `-- 01_quy_che_dao_tao/
|   `-- processed/            # Tầng 2: Markdown đã chuẩn hóa heading
|       `-- 01_quy_che_dao_tao/
|-- docs/                     # Thư mục tài liệu, phân tích, kế hoạch và báo cáo chi tiết
|   |-- bao_cao_cai_tien_bai_nghien_cuu.md  # Báo cáo cải tiến bài nghiên cứu
|   |-- bao_cao_cai_tien_rag.md             # Báo cáo cải tiến hiệu năng hệ thống RAG
|   |-- bao_cao_danh_gia_du_lieu_raw.md     # Báo cáo đánh giá chất lượng dữ liệu thô
|   |-- bao_cao_mo_rong_corpus.md           # Báo cáo phương án mở rộng ngữ liệu
```

```
|  |-- bao_cao_phan_tich_yeu_cau.md        # Báo cáo phân tích yêu cầu hệ thống
|  |-- de_xuat_code_cai_tien_pipeline.md   # Đề xuất mã nguồn cải tiến pipeline
|  |-- de_xuat_xu_ly_nhan_xet.md           # Kế hoạch tiếp thu và xử lý nhận xét
|  |-- giao_trinh/                         # Tài liệu giáo trình và lý thuyết RAG
|  |  `-- co_so_ly_thuyet_rag.html         # Cơ sở lý thuyết RAG (HTML)
|  |-- huong_dan_src.md                    # Hướng dẫn sử dụng mã nguồn src/
|  |-- ke_hoach_them_du_lieu_dao_tao.md    # Kế hoạch bổ sung dữ liệu đào tạo
|  |-- kha-task-1.md                       # Tài liệu giao việc Task 1
|  |-- llamaindex_tong_quan.html           # Tổng quan về thư viện LlamaIndex
|  |-- nhanxet.md                          # Tổng hợp nhận xét và góp ý
|  |-- phan-cong-4-nguoi.md                # Bảng phân công nhiệm vụ nhóm 4 người
|  |-- phan_loai_tai_lieu_va_huong_giai_quyet.md # Phân loại tài liệu & hướng giải quyết
|  |-- phan_tich_ky_thuat_nang_cao_va_co_hoi_cai_tien.md # Phân tích kỹ thuật nâng cao
|  |-- task-0.md                           # Chi tiết công việc Task 0
|  `-- task-tap-kiem-chung-va-phan-cong.md # Phân công tập kiểm chứng dữ liệu
|-- eval/                                  # Thư mục đánh giá hiệu năng hệ thống
|  |-- results/                            # Thư mục lưu kết quả đánh giá (JSON, Markdown)
|  |-- baselines.py                        # Baseline BM25 và không xếp hạng
|  |-- cau_hoi_danh_gia.json               # Bộ câu hỏi có đáp án chuẩn
|  |-- compare_results.py                  # Script so sánh kết quả các lần chạy
|  |-- dev_questions_dot1.json# Bộ câu hỏi phát triển đợt 1
|  |-- dev_questions_dot3.json# Bộ câu hỏi phát triển đợt 3
|  |-- dev_questions_dot2.json# Bộ câu hỏi phát triển đợt 2
|  |-- evaluate_answers.py                 # Chấm câu trả lời bằng evaluator của LlamaIndex
|  |-- evaluate.py                         # Chạy đánh giá truy hồi
|  |-- metrics.py                          # Đo độ chính xác và kiểm định thống kê
|  |-- so_sanh_langchain.py                # Dựng lại phần truy hồi bằng LangChain để so sánh (Mục 1.3.2)
|  |-- run_batch_cau_hoi.py                # Script chạy batch câu hỏi tự động
|  |-- test_kiem_chung_3183.json           # File JSON kiểm chứng QĐ 3183
|  |-- test_kiem_chung_ngoai.json # 20 câu ngoài phạm vi của tập kiểm chứng
|  |-- test_kiem_chung_nhan_B.json # Nhãn do người thứ hai gán độc lập
|  |-- test_kiem_chung_qc2021.json# File JSON kiểm chứng quy chế 2021
|  |-- test_questions_01.json # Bộ câu hỏi kiểm thử 01
|  |-- tra_loi_dev.py                      # Sinh câu trả lời bộ dev kèm token, độ trễ lấy từ Ollama
|  `-- test_questions_02.json # Bộ câu hỏi kiểm thử 02
```

```
|-- report/                  # Mã nguồn và tài liệu báo cáo LaTeX
|   |-- appendices/          # Các phụ lục báo cáo (Nhật ký, minh chứng, tự kiểm, thực nghiệm)
|   |   |-- a-nhat-ky-tuan.tex
|   |   |-- b-minh-chung-nhom.tex
|   |   |-- c-tu-kiem.tex
|   |   `-- d-thuc-nghiem.tex
|   |-- chapters/            # Nội dung các chương báo cáo
|   |   |-- 03-moi-truong-kien-truc.tex
|   |   |-- 04-cong-nghe-cot-loi.tex
|   |   |-- 05-node-parser.tex
|   |   |-- 06-persistent-index.tex
|   |   |-- 07-trich-dan-tu-choi.tex
|   |   |-- 08-nang-cao.tex
|   |   |-- 09-phan-tich-thiet-ke.tex
|   |   |-- 10-trien-khai.tex
|   |   |-- 11-danh-gia-nhin-lai.tex
|   |   |-- 12-huong-phat-trien.tex
|   |   |-- p1-01-gioi-thieu-llamaindex.tex
|   |   `-- p1-02-vi-the.tex
|   |-- config/              # Cấu hình định dạng và macros LaTeX
|   |   |-- macros.tex
|   |   |-- metadata.tex
|   |   `-- preamble.tex
|   |-- figures/             # Hình ảnh và tài nguyên đồ họa báo cáo
|   |   `-- logoITSGU.png
|   |-- frontmatter/         # Phần mở đầu (Bìa, lời mở đầu)
|   |-- main.tex             # File mã nguồn LaTeX chính để biên dịch báo cáo
|   `-- main.pdf             # File báo cáo PDF đã biên dịch hoàn chỉnh
|-- scripts/                 # Các script tiện ích và tự động hóa
|   |-- check_model_memory.py  # Kiểm tra bộ nhớ VRAM của mô hình
|   |-- clean_md.py          # Làm sạch các tệp Markdown
|   |-- colab_server.py      # Chạy server trên Google Colab
|   |-- extract_to_md.py     # Trích xuất văn bản sang Markdown
|   |-- hello_world.py       # Script kiểm tra nhanh môi trường
|   |-- manager.py           # Quản lý các tác vụ hệ thống
```

```
|  |-- pull_models.sh        # Script tải các mô hình cần thiết
|  |-- reorganize_raw.py     # Sắp xếp lại cấu trúc dữ liệu thô
|  |-- setup_server.sh       # Cài đặt server phiên bản 1
|  |-- setup_server_v2.sh    # Cài đặt server phiên bản 2
|  `-- test_ocr.py           # Kiểm thử tính năng OCR
|-- src/                     # Mã nguồn lõi của hệ thống RAG
|  |-- bm25.py               # BM25 thuần Python phục vụ hybrid search
|  |-- config.py             # RAGConfig, 2 profile và resolve_device()
|  |-- corpus.py             # Danh sách văn bản nạp vào index
|  |-- index_builder.py      # Node, VectorStoreIndex và fingerprint
|  |-- phien_ban.py          # Lọc phiên bản: hiệu lực, khóa áp dụng, đưa bản sửa đổi lên trước
|  |-- model_server.py       # FastAPI giữ embedding và reranker trên GPU
|  |-- processed_loader.py   # Chuyển Markdown Tầng 2 thành TextNode
|  |-- query_engine.py       # Prompt, LLM (temperature=0, seed) và nguồn
|  |-- remote_models.py      # Client gửi request đến model server
|  `-- retriever.py          # Vector, BM25, rerank và ngưỡng tương đồng
|-- tests/                   # Thư mục kiểm thử tự động hệ thống (Unit tests)
|  |-- __init__.py           # Khởi tạo gói kiểm thử
|  |-- test_bm25.py          # Kiểm thử thuật toán BM25
|  |-- test_corpus.py        # Kiểm thử nạp và quản lý ngữ liệu
|  |-- test_llm.py           # Kiểm thử tương tác với mô hình ngôn ngữ
|  |-- test_luat_cham.py     # Kiểm thử luật chấm truy hồi
|  |-- test_metrics.py       # Kiểm thử các chỉ số đánh giá
|  |-- test_phien_ban.py     # Kiểm thử lọc phiên bản
|  `-- test_processed_loader.py # Kiểm thử bộ nạp dữ liệu đã xử lý
|-- .env.example             # Mẫu cấu hình biến môi trường
|-- .gitignore               # Danh sách các file bỏ qua không commit (storage)
|-- Makefile                 # Các lệnh nhanh: test, eval, baseline, report
|-- README.md                # Tài liệu hướng dẫn cài đặt và sử dụng dự án
`-- requirements.txt         # Danh sách thư viện Python đã ghim phiên bản
```

Mọi script chạy từ thư mục gốc của repository, ví dụ RAG_PROFILE=server python src/pipeline.py. Các module trong src/ vừa là thư viện (được module sau import) vừa có khối __main__ để chạy thử riêng từng giai đoạn.

3.3.2 Cấu hình tập trung và hai profile

Toàn bộ tham số RAG nằm trong src/config.py dưới dạng một dataclass RAGConfig. Có hai profile sẵn: local (máy cá nhân, mô hình nhỏ) và server (mô hình lớn hơn, có reranker, LLM chạy trên máy GPU thuê). Profile được chọn bằng biến môi trường RAG_PROFILE (đọc cả từ file .env), mặc định là local; giá trị không hợp lệ làm hàm get_config() ném ValueError.

Đoạn mã 3.4: Khai báo dataclass RAGConfig trong src/config.py.

```
@dataclass
class RAGConfig:
    embed_model_name: str
    embed_device: str  # "cpu" or "cuda"
    llm_model_name: str
    ollama_base_url: str
    request_timeout: float = 180.0
    temperature: float = 0.0       # 0 = tat dinh (sua loi tra loi khong on dinh)
    seed: int = 42
    reranker_model_name: str | None = None
    reranker_top_n: int = 3
    reranker_max_length: int = 2048  # token mỗi cặp (câu hỏi, Node)
    use_rerank: bool = False       # profile server đặt True
    loc_phien_ban: bool = True     # src/phien_ban.py
    similarity_top_k: int = 10
    similarity_cutoff: float | None = None   # nguong tuong dong (None = tat)
    use_hybrid: bool = False              # BM25 + vector qua QueryFusionRetriever
    bm25_top_k: int = 10
    fusion_mode: str = "reciprocal_rerank"
    response_mode: str = "compact"
    streaming: bool = True
    persist_dir: str = ""          # trong = storage/<profile>/<corpus>
```

Bảng 3.2: Hai profile cấu hình trong src/config.py. Số liệu truy hồi trong báo cáo đo trên GPU RTX A2000 4 GB ở fp16 (ngày 17/10/2026).

Tham số	Profile local	Profile server
Embedding	bkai-foundation-models/ vietnamese-bi-encoder (768 chiều)	AITeamVN/Vietnamese_Embedding (1024 chiều)
Thiết bị embedding	cuda (mặc định; resolve_device() tự hạ về cpu khi không có GPU)	cuda (mặc định; resolve_device() tự hạ về cpu khi không có GPU)

Tham số	Profile local	Profile server
LLM (Ollama)	qwen3:4b, localhost:11434	qwen3:8b, địa chỉ lấy từ OLLAMA_BASE_URL
Reranker	không dùng	AITeamVN/Vietnamese_Reranker, đọc 2 048 token, giữ top 3 (bật mặc định, sau bước lọc phiên bản)
Retriever	top-10	top-10

Chỉ mục được lưu tách theo profile và tự build lại khi cấu hình đổi (Chương 6).

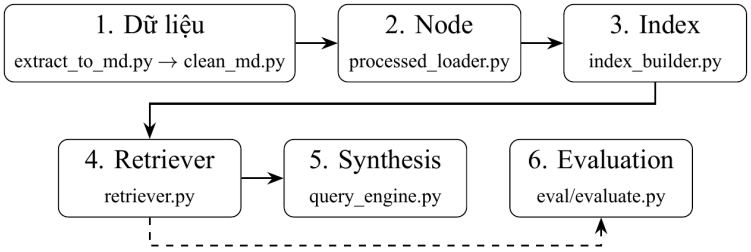
Nguồn dữ liệu thực tế được khai báo ở *một* chỗ: src/corpus.py (CORPUS_DIR và dict SGU_HIEU_LUC). Trường corpus_sources cũ cùng hai file PDF không tồn tại đã được gỡ bỏ, nên không còn tình trạng cấu hình trỏ vào file không có thật.

3.3.3 Các script hỗ trợ

- scripts/test_ocr.py: script thử OCR ban đầu, OCR một file PDF ra data/ocr; không còn nằm trong pipeline. Đầu vào thật của chỉ mục đi qua scripts/extract_to_md.py rồi scripts/clean_md.py (Mục 4.1.2).
- scripts/check_model_memory.py: đo RAM/VRAM khi nạp một mô hình embedding hoặc reranker (nhận repo_id, --type, --device), giúp chọn mô hình vừa với phần cứng.
- scripts/pull_models.sh: tải LLM qua Ollama và tải embedding/reranker từ HuggingFace theo profile, đồng bộ với config.py; dừng với thông báo lỗi nếu thiếu ollama, daemon chưa chạy hoặc thiếu huggingface_hub.
- scripts/manager.py: công cụ dòng lệnh gom các thao tác với dịch vụ GPU thuê (xem, thuê, khởi động lại, xóa máy; xem số dư), gọi thử Ollama và chạy một server embedding.
- scripts/setup_server_v2.sh, scripts/colab_server.py: dựng phần chạy trên GPU (Ollama + model server embedding/reranker) trên Google Colab hoặc máy thuê, mở URL công khai qua Cloudflare quick tunnel và in sẵn các dòng OLLAMA_BASE_URL, MODEL_SERVER_URL để dán vào .env ở máy local. Thư viện của model server được ghim cùng phiên bản với requirements.txt, để vector sinh trên Colab khớp với chỉ mục đã build ở local. Nhờ vậy phần đánh giá câu trả lời — vốn cần LLM — chạy được mà không cần máy có GPU.

- eval/run_batch_cau_hoi.py: chạy hàng loạt danh sách câu hỏi trong eval/results/cau-hoi01.md qua query engine, ghi câu trả lời kèm nguồn ra cau-hoi01-result.md để chấm tay (Mục 10.2.2.4).
- Makefile: gom make test (54 kiểm thử; 10 kiểm thử cần Ollama tự bỏ qua khi chạy offline), make eval (đo các cấu hình truy hồi: vector, rerank, lọc phiên bản, hybrid), make baseline, make eval-answers và make report.
- .github/workflows/ci.yml: CI cài đúng phần tối thiểu (không torch, không model) và chạy toàn bộ kiểm thử offline trên mỗi lần push.

Chuỗi xử lý của đồ án gồm sáu giai đoạn. Hình 3.1 thể hiện chuỗi này và file mã nguồn tương ứng; cả sáu giai đoạn đều đã có mã chạy được. Pha chuẩn bị (1–3) chạy một lần và lưu chỉ mục xuống đĩa; pha truy vấn (4–5) chạy mỗi câu hỏi; giai đoạn 6 dùng lại retriever của giai đoạn 4.



Hình 3.1: Sáu giai đoạn của pipeline RAG và file mã nguồn tương ứng; mũi tên nét đứt: bộ đánh giá gọi trực tiếp retriever.

Bảng 3.3: Trạng thái mã nguồn của từng giai đoạn.

Giai đoạn	**Mức**	**File**	**Chạy được**
1. Dữ liệu	1A (bắt buộc lõi)	scripts/extract_to_md.py, scripts/clean_md.py	Có
2. Node	1A (bắt buộc lõi)	src/processed_loader.py	Có
3. Index	1A (bắt buộc lõi)	src/index_builder.py	Có
4. Retriever	1A (bắt buộc lõi)	src/retriever.py	Có
5. Response Synthesis	1A (bắt buộc lõi)	src/query_engine.py, src/pipeline.py	Có
6. Evaluation	1A (bắt buộc lõi)	eval/evaluate.py, eval/baselines.py, eval/evaluate_answers.py	Có (truy hồi: tự động; câu

Giai đoạn	Mức	File	Chạy được
			trả lời: cần Ollama)
Hybrid BM25	Tầng 3	src/bm25.py, src/retriever.py	Có (mặc định tắt)
Rerank	Tầng 3	src/retriever.py	Có (profile server, mặc định bật, đọc 2 048 token, đi sau src/phien_ban. py)
Model server	Tầng 3	src/model_server.py, src/remote_models.py	Có
Kiểm thử	Tầng 2	tests/, Makefile, CI	Có (54 kiểm thử)
Giao diện web	1B	app/app.py	Có (chạy thử)

3.4 Kiểm chứng môi trường

Chỉ mục khớp với loader. Chạy src/index_builder.py (profile server, GPU fp16, ngày 17/10/2026):

Đoạn mã 3.5: Kiểm chứng số Node trong chỉ mục.

```
[INFO] Load Index từ storage/server/01_quy_che_dao_tao
Index: storage/server/01_quy_che_dao_tao | số node trong Index: 118 | từ processed_loader: 118 | khớp: True
```

Kết nối Ollama. Một câu trả lời thật trong nhật ký Gradio (eval/results/questions_log.jsonl, ngày 08/10, qwen3:8b) cho câu "Sinh viên được rút học phần trong thời gian nào?":

Đoạn mã 3.6: Câu trả lời mẫu qua Ollama.

```
Theo quy định tại Điều 7 — Quy chế đào tạo trình độ đại học SGU 2021 (sửa đổi bởi
QĐ 3183/2025), việc rút bớt học phần trong khối lượng học tập đã đăng ký được thực
hiện trong 03 tuần đầu tiên của học kì.
```

Câu trả lời khớp khoản 4 mục sửa đổi Điều 7 của QĐ 3183/2025.

CHƯƠNG 4. BẢN CHẤT CÔNG NGHỆ: BẮT BUỘC LÕI

Chương này đi qua sáu mục lõi (Tầng 1A). Mỗi mục viết theo chuỗi: *cơ chế → mã nguồn → kết quả chạy → ý nghĩa trong đồ án*. Số liệu sinh lại được bằng make test và make eval trên đúng phiên bản thư viện đã ghim (Chương 3).

Bảng 4.1: Sáu mục lõi và minh chứng tương ứng.

TT	Yêu cầu lõi	Cơ chế	Minh chứng (file / log)	Vị trí trong đồ án
1	Ingestion, metadata	OCR bản scan, Markdown hai tầng, front matter	scripts/extract_to_md.py, scripts/clean_md.py; 20 văn bản	Nguồn dữ liệu duy nhất
2	Node/chunk, chunk size và overlap	TextNode, cắt theo Điều, SentenceSplitter 1024/50	src/processed_loader.py; 118 Node	Đơn vị truy hồi và trích dẫn
3	Embedding + vector index	HuggingFace embedding, VectorStoreIndex, persist	src/index_builder.py; 118/118 khớp	Nền cho truy hồi
4	Retriever top-k	Cosine, top-k = 10	src/retriever.py; Bảng 4.2	Chọn ngữ cảnh cho LLM
5	Response synthesis + truy vết nguồn	RetrieverQueryEngine, chế độ compact, prompt riêng	src/query_engine.py; dòng “Nguồn”	Sinh câu trả lời
6	Đánh giá thực nghiệm	Recall@k, MRR, nDCG, khoảng tin cậy	eval/evaluate.py; R@1 91,2% trên 91 câu dev (Chương 8)	Chứng minh chất lượng truy hồi

4.1 Nạp dữ liệu (Ingestion)

4.1.1 Vì sao cần OCR

Bản PDF của Quy chế đào tạo SGU là bản scan: mỗi trang là một ảnh, không có lớp văn bản (text layer). Khi đọc bằng bộ đọc PDF thông thường, hệ thống gần như không lấy được chữ nào. Để có văn bản, cần OCR [12].

Tesseract [12] là công cụ OCR mã nguồn mở, có gói ngôn ngữ tiếng Việt (vie); trong Python được gọi qua thư viện pytesseract. Vì Tesseract nhận ảnh làm đầu vào, mỗi trang PDF trước hết được render thành ảnh bằng pdf2image (dựa trên poppler). Độ phân giải càng cao thì nhận dạng càng tốt nhưng chậm hơn; đồ án dùng 300 dpi.

OCR tiếng Việt luôn có sai sót, nhất là với dấu thanh và chữ in mờ (ví dụ nhầm "NHÂN" thành "NHẦN"). Điều này ảnh hưởng đến các bước sau, đặc biệt khi quy tắc cắt đoạn dựa vào việc nhận đúng từ "Điều". Mục 4.1.3 nêu chỉ số chất lượng OCR quan sát được.

4.1.2 Cơ chế

LlamaIndex có sẵn SimpleDirectoryReader để đọc PDF, nhưng các văn bản quy chế của SGU là bản scan: trang PDF chỉ là ảnh, không có lớp văn bản nên bộ đọc thường trả về gần như rỗng. Vì vậy đồ án chuẩn bị dữ liệu ngoài LlamaIndex rồi mới đưa vào, theo hai tầng:

1. **Tầng 1 — scripts/extract_to_md.py**: nếu PDF có lớp văn bản thì trích trực tiếp; nếu không thì render từng trang ở 300 dpi bằng pdf2image rồi OCR bằng pytesseract với lang="vie" [12]. Script lưu kết quả vào data/ocr/, kèm chỉ số chất lượng là *tỷ lệ âm tiết tiếng Việt hợp lệ*; khi một văn bản có nhiều bản scan, chỉ số này dùng để chọn bản tốt hơn.
2. **Tầng 2 — scripts/clean_md.py**: làm sạch văn bản tầng 1 và dựng lại cấu trúc bằng heading Markdown: # cho tên văn bản, ## cho Chương, ### cho Điều, #### cho mục sửa đổi. Kết quả nằm ở data/processed/.

Tầng 1 là nơi *duy nhất* được sửa tay; tầng 2 luôn sinh lại được từ tầng 1, nên không có hai bản chân lý mâu thuẫn nhau. Cách tách này cũng là điều kiện để dùng lại toàn bộ hạ tầng LlamaIndex phía sau: mọi thứ từ node trở đi đều đi trên Markdown có cấu trúc.

Metadata. Mỗi file tầng 2 có khối front matter gồm ten_van_ban, file_goc, url, so_hieu, ngay_ban_hanh, tinh_trang và các khóa hiệu lực: sua_doi, bi_sua_doi_boi, thay_the, bi_thay_the_boi (quan hệ giữa các văn bản), ap_dung_khoa_tu, ap_dung_khoa_den (khóa tuyển sinh áp dụng) và ap_dung_khoa_tu_theo_dieu (khi mỗi Điều của văn bản sửa đổi áp dụng từ khóa khác nhau, như QĐ 3183). processed_loader chuyển chúng thành metadata của Node: nhóm đầu là nền cho trích dẫn nguồn ở Mục 4.5, nhóm hiệu lực dùng cho bước lọc phiên bản (Mục 8.2) và không đi vào embedding hay prompt.

4.1.3 Kết quả chạy thật

Corpus hiện tại gồm 20 văn bản đang áp dụng (ngày 17/10/2026). Chạy python src/processed_loader.py cho:

Đoạn mã 4.1: Số Node theo từng văn bản (sinh lại được bằng make test).

```
118 Node từ 20 văn bản trong data/processed/01_quy_che_dao_tao
  1  02.-Tiethoc_phonghoc.md
  1  202111_2366-TB-DHSG_ to chuc day hoc cac hoc phan tieng Anh ... 2021 - 2022.md
  5  2022-2375-QyD Chuyen Chuong Trinh Dao Tao DoiVoiSinhVenDHCQ.md
  1  22_quy-dinh-ve-khoi-luong-dao-tao-cua-cac-he-dao-tao-chinh-quy-dao-tao.md
 25  25_quy-che-dao-tao-trinh-do-dai-hoc-tai-truong-dai-hoc-sai-gon.md
  5  29_2101-Qui dinh vv to chuc day hoc cac HP tieng Anh ... tu nam 2024 tro di.md
  1  30_TB so 2451 vv bo sung chung chi TA quy doi ... khoa 2021 tro ve truoc.md
  1  3260-QD-Hoan-thi-.md
 13  8_QD810QuyDinhVeHocCungLucHaiChuongTrinhDoiVoiSVDHCQ.md
  8  CV2626ToChucDayHocCacHPTAKoChuyen...TuNam2022TroDi.md
  1  CV_3248_001_khieunai.md
  5  HD-Quy trinh phuc khao bai thi ket thuc hoc phan.md
  9  QD 3183.2025 - Sua doi Quy che Dao tao trinh do DH_LienQuanRutMonHoc.md
 17  QD898BanHanhQuyDinhToChucThiHocKi.md
  1  QuyDinh_CapBanSao.md
  1  QuyDinh_CapBangDiem.md
  1  Quydinh150tc.md
 17  QĐ_CVHT.md
  1  TB so 2567- Bo sung chung chi tieng Anh PTE Academic ... TA khong chuyen.md
  4  TB_ChuanDauRa_2020.md
```

Văn bản dài nhất (Quy chế 2021) cho 25 Node: 21 Điều của Quy chế cộng 4 Node thuộc phần "Quyết định ban hành". 10 văn bản không chia Điều (thông báo, quy định ngắn) cho mỗi văn bản một Node, đúng như thiết kế MIN_TU_NOI_DUNG (xem Mục 4.2). QĐ 898/2025 và quy chế cố vấn học tập cho 17 Node mỗi văn bản vì có Điều dài bị SentenceSplitter cắt tiếp.

Chất lượng OCR không đều: văn bản quan trọng nhất (QĐ 3183/2025) có tỷ lệ âm tiết hợp lệ thấp nhất (94,9%) và vẫn còn lỗi như "Sửa đổi", "quy đổi" (xem trích đoạn ở Mục 4.5.2).

4.1.4 Ý nghĩa trong đồ án

Toàn bộ hệ thống chỉ biết những gì đi qua bước này. Vì OCR sai được sửa tay ở tầng 1 và tầng 2 sinh lại tự động, chất lượng dữ liệu cải thiện dần mà không phải chạy lại OCR. Lý do không đóng gói mỗi trang PDF thành một Document được trình bày ở Chương 5.

4.1.5 Mở rộng dữ liệu đợt 2–3

Ở tuần 6, corpus được mở rộng từ 6 lên 20 văn bản theo kế hoạch trong docs/ke_hoach_them_du_lieu_dao_tao.md. Nhóm mở từng tệp trong 49 tệp thuộc nhóm Đào tạo để phân loại: bỏ bản trùng, bản hết hiệu lực và biểu mẫu; văn bản xung đột với Quy chế 2021 (QĐ 736 về thời gian tối đa) không nạp mà giữ làm ca kiểm thử xung đột phiên bản. Khó khăn chính nằm ở chỗ các văn bản có 7 định dạng khác nhau (Bảng 4.5), trong khi bộ cắt ban đầu chỉ hiểu dạng Chương, Điều và mục sửa đổi.

Bảng 4.5: Bảy định dạng văn bản trong nhóm dữ liệu Đào tạo và cách xử lý trong đồ án.

Mã	Định dạng	Ví dụ trong corpus	Cách xử lý
F1	Quy chế, quy định chia Chương và Điều	Quy chế 2021, QyĐ 2626, QyĐ 2101, QĐ 810	Một Điều là một Node
F2	Quyết định ban hành kèm văn bản	QĐ 898/2025, quy chế cố vấn học tập	Phần quyết định tách riêng (phan = ban_hanh); hiệu lực và văn bản bị thay thế ghi vào front matter
F3	Quy định đánh số mục, không có Điều	QĐ 3260 về hoãn thi, quy định cấp bảng điểm, cấp bản sao	Một Node mỗi văn bản ngắn; văn bản dài cắt tiếp bằng SentenceSplitter
F4	Thông báo mà nội dung chính là bảng	TB 2366, TB 2451, TB 2567, tiết học, TB 536/2020	Bảng chép tay thành bảng Markdown vì OCR làm hỏng số trong bảng (“§ § §” thay cho “8 8 8”); bảng quy đổi chép thêm ra CSV

F5	Văn bản sửa đổi, bổ sung, thay thế	QĐ 3183, TB 2451, TB 2567, QĐ 898	Như văn bản gốc, cộng quan hệ sua_doi, thay_the và khóa áp dụng trong front matter
F6	Hướng dẫn quy trình theo bước	HD 1181 về phúc khảo	Chép thành bảng các bước; quy trình dài được cắt thành 5 đoạn
F7	Công văn ngắn 1–2 trang	Quy định 150 tín chỉ, Quy định 3248/2018	Một Node mỗi văn bản

Xác minh hiệu lực. Với mỗi văn bản đang nạp, số hiệu được nhập tay và đối chiếu với PDF (OCR từng đọc 3089 thành 3084), ngày ban hành ghi đủ ngày, tháng, năm, rồi đọc phần căn cứ và điều khoản thi hành để xác định tình trạng hiệu lực, quan hệ sửa đổi hoặc thay thế và khóa tuyển sinh áp dụng. Nguồn xác minh ghi trong _bao_cao_kiem_tra.md của từng thư mục và tổng hợp ở _metadata_van_ban.csv. Kết quả xác minh nay là đầu vào của bước lọc phiên bản (Mục 8.2). Ví dụ: Quy định 3248/2018 bị QĐ 898/2025 sửa đổi (lệ phí phúc khảo đổi từ 100 000 đồng thành 150 000 đồng); QĐ 898 thay thế QyĐ 3089/2020 nên văn bản 3089 không được nạp; chuỗi quy đổi tiếng Anh gồm TB 2366 và TB 2451 (khóa 2021 trở về trước), QyĐ 2626 (khóa 2022, 2023), QyĐ 2101 (từ khóa 2024), và TB 2567 bổ sung chứng chỉ PTE cho các nhóm khóa đó.

Bảng quy đổi dạng CSV. Năm bảng quy đổi chứng chỉ tiếng Anh được chép từ PDF sang data/bang_quy_doi/ (300 dòng, 8 cột: chứng chỉ, mức từ, mức đến, học phần, điểm, khóa áp dụng từ, khóa áp dụng đến, trang nguồn). CSV này là đáp án đối chiếu cho câu hỏi quy đổi và là dữ liệu cho công cụ tra bảng dự kiến (Mục 12.1). Lần đối chiếu từng dòng với PDF sửa 24 ô của TB 2366: hàng điểm 9 trong văn bản dùng bất đẳng thức chặt (450 < Nghe và Đọc < 600) nhưng bản chép ghi như khoảng có cả hai đầu, khiến điểm 450 và 600 khớp hai mức cùng lúc. Lần đối chiếu cũng ghi nhận hai điểm của chính văn bản gốc mà công cụ tra bảng phải xử lý: QyĐ 2626 tự chồng biên (Linguaskill 140–149 và 149–158), và TOEIC 4 kỹ năng là điều kiện phải đạt đồng thời cả Nghe và Đọc, Nói, Viết (biên bản ở data/bang_quy_doi/doi_chieu_D4.md).

4.2 Chia Node/chunk

4.2.1 Cơ chế

Module src/processed_loader.py đọc Markdown tầng 2 và sinh TextNode — đối tượng đơn vị của LlamaIndex [1]:

1. Tách file theo heading bằng biểu thức chính quy (?m)^(#{1,4}) (lookahead giữ lại dòng heading), rồi theo dõi cấp heading để biết Node thuộc văn bản nào, Chương nào.
2. Mỗi ### (Điều) hoặc #### (mục sửa đổi) là một Node; nhãn dieu hoặc sua_doi_dieu được trích bằng Điều (\d+).
3. Nội dung nằm ngay dưới #/## chỉ thành Node khi dài từ MIN_TU_NOI_DUNG = 30 từ trở lên (dành cho văn bản không chia Điều).
4. Node dài hơn MAX_TU = 1 024 từ được cắt tiếp bằng SentenceSplitter(chunk_size=1024, chunk_overlap=50) và gắn thêm phan_doan. Hai ngưỡng này khác đơn vị: MAX_TU đếm theo từ tách bằng khoảng trắng, còn SentenceSplitter đếm theo token tiktoken. Tiếng Việt trung bình 2,21 token tiktoken mỗi từ, nên mỗi đoạn sau khi cắt dài khoảng 460 từ chứ không phải 1 024. Trên corpus hiện tại, 19 Node mang phan_doan, cắt từ 5 Điều hoặc văn bản dài (HD 1181, QĐ 898 Điều 3, TB 536, QyĐ 2626 Điều 3, quy chế cố vấn học tập Điều 6).

4.2.2 Chunk size và overlap ở mức ứng dụng

- **Đơn vị chính là Điều, không phải số token.** Một Điều của quy chế là một đơn vị nghĩa trọn vẹn (một thủ tục, một thang điểm). Cắt cố định theo token dễ chia đôi bảng điểm hoặc tách điều kiện khỏi hệ quả. Vì vậy chunk_size chỉ là *trần an toàn*: chỉ Điều dài hơn MAX_TU = 1 024 từ mới bị cắt tiếp.
- **Vì sao 1 024.** Mô hình embedding của profile server (AITeamVN/Vietnamese_Embedding) nhận tới 8 192 token, còn Node dài nhất của corpus là 1 210 token, nên không Node nào bị cắt khi embed. Điều này không đúng với profile local: vietnamese-bi-encoder chỉ nhận 256 token, 63/118 Node dài hơn ngưỡng đó và mất phần cuối khi embed (Phụ lục C). Độ dài Node đo ngày 17/10 ở Bảng 4.6. Độ dài cũng quyết định số lượt gọi LLM: văn bản LLM nhận

trung bình 738 token tiktoken mỗi Node, nên 10 Node trung bình khoảng 7 381 token, sát hạn mức khoảng 7 400 token của chế độ compact (context_window 8 192 trừ phần trả lời và prompt). Câu nào truy hồi trúng các Điều dài sẽ bị chia thành hai lượt gọi, và lượt sau dùng prompt refine mặc định tiếng Anh, không có các ràng buộc của QA_PROMPT. Với rerank giữ 3 Node, ba Node dài nhất cộng lại là 6 226 token, luôn vừa một lượt.

- **Overlap 50.** Chỉ có tác dụng khi một Điều bị cắt tiếp: phần chồng lấp giữ câu dẫn của khoản trước để đoạn sau không mất ngữ cảnh.
- **Đánh đổi.** Node to (cả Điều) giữ ngữ cảnh nhưng "pha loãng" vector và khiến LLM đọc sai chi tiết số trong Điều dài (lỗi "30 tuần" ở Chương 10); Node nhỏ (theo khoản) khớp chính xác hơn nhưng mất câu dẫn. Hướng dung hòa là small-to-big (Chương 12).

Chưa triển khai. Thí nghiệm so sánh các cách chunk (cố định 256/512/1024 token, theo câu, theo Điều, theo khoản) bằng Recall@1/3/5 và MRR trên cùng bộ câu hỏi chưa chạy. Đây là việc của giai đoạn GĐ3.

Bảng 4.6: Độ dài mỗi Node (văn bản đem embed) trên corpus 118 Node, đo ngày 17/10/2026 bằng load_corpus_nodes() và tokenizer của từng mô hình.

Đơn vị đo	**Trung bình**	**Trung vị**	**Lớn nhất**
Ký tự	1 352	1 108	4 641 (Điều 4 QĐ 898)
Từ (tách theo khoảng trắng)	295	238	1 032
Token tiktoken cl100k (mặc định của LlamaIndex)	651	536	2 128
Token AITeamVN/Vietnamese_Embedding (cùng tokenizer với reranker)	372	275	1 210
Token vietnamese-bi-encoder	379	285	1 213

(profile local)			
Token Qwen3-8B (LLM)	428	335	1 402

4.3 Embedding và lập chỉ mục

4.3.1 Mô hình embedding tiếng Việt được chọn

Chất lượng truy hồi phụ thuộc mạnh vào việc mô hình embedding có được huấn luyện cho tiếng Việt hay không. Theo cấu hình trong src/config.py, đồ án dùng:

- Profile chạy cục bộ: bkai-foundation-models/vietnamese-bi-encoder, vector 768 chiều [13];
- Profile máy chủ: AITeamVN/Vietnamese_Embedding, vector 1024 chiều [14].

Cả hai đều được nạp qua llama-index-embeddings-huggingface (lớp HuggingFaceEmbedding, mặc định chuẩn hóa vector về độ dài 1 nên tích vô hướng chính là cosine). Vì hai mô hình cho vector khác số chiều, chỉ mục đã lập bằng mô hình này không dùng được với mô hình kia; đổi mô hình embedding đồng nghĩa phải lập lại chỉ mục. Phụ lục C ghi lại so sánh hai mô hình.

4.3.2 Cơ chế

Module src/index_builder.py làm ba việc:

1. **Nạp Node** bằng load_corpus_nodes() của processed_loader.
2. **Đặt mô hình embedding** vào Settings.embed_model — Settings là cấu hình toàn cục của LlamaIndex, mọi thành phần cần embedding đều lấy từ đây. Hàm resolve_device() tự hạ cấp về CPU khi máy không có CUDA, nhờ đó cùng một mã nguồn chạy được trên máy GPU, trên CPU và trên CI.
3. **Dựng hoặc nạp lại** VectorStoreIndex. Chỉ mục được dựng thẳng từ danh sách TextNode đã cắt sẵn chứ không từ Document: nếu truyền Document, LlamaIndex sẽ tự cắt bằng SentenceSplitter mặc định và làm mất ranh giới Điều — đây chính là lý do phải gọi constructor bằng danh sách Node.

4.3.3 Kết quả chạy thật

Với profile server (AITeamVN/Vietnamese_Embedding, 1 024 chiều) trên corpus 20 văn bản, lần build lại ngày 16/10/2026 ghi ra:

Đoạn mã 4.2: Dòng log khi build lại chỉ mục (GPU 4 GB, fp16).

```
Corpus, model embedding hoặc loader đã đổi so với index đã lưu — build lại
Generating embeddings: 100%|##########| 118/118 [00:04<00:00, 28.78it/s]
Index: storage/server/01_quy_che_dao_tao | 118 Node
```

Chỉ mục lưu theo storage/<profile>/<đường dẫn corpus>, nên hai profile có số chiều vector khác nhau không còn ghi đè lên nhau. Embedding 118 Node mất khoảng 4 giây trên GPU 4 GB ở fp16; trên CPU, 54 Node đã mất 77 giây (đo ngày 09/10).

Cơ chế lưu và nạp lại chỉ mục, cùng fingerprint chống dùng nhầm chỉ mục cũ, được trình bày ở Chương 6.

4.4 Truy hồi (Retriever)

4.4.1 Cơ chế

Hàm get_retriever() trong src/retriever.py gọi index.as_retriever() với tham số similarity_top_k = 10: câu hỏi được embedding bằng đúng mô hình đã dùng khi lập chỉ mục, rồi so cosine với toàn bộ Node (SimpleVectorStore tìm vét cạn, đủ nhanh với vài chục đến vài trăm Node) và trả về 10 Node điểm cao nhất. Hai tầng tuỳ chọn (trộn BM25 và xếp hạng lại) thuộc phần nâng cao (Chương 8).

4.4.2 Kết quả chạy thật: top-5 kèm điểm

Chạy ngày 09/10 trên corpus 54 Node, profile server, CPU, vector không rerank. Bảng chỉ minh họa cơ chế; số liệu trên corpus hiện tại ở Mục 4.6 và Chương 8:

Bảng 4.2: Top-5 Node của ba câu hỏi mẫu.

Hạng	**Cosine**	**Node**
"Một năm học có bao nhiêu học kỳ?"		
1	0,4377	QĐ 3183/2025 – sửa đổi Điều 6
2	0,4336	Quy chế 2021 – Điều 6
3	0,3713	Quy chế 2021 – Điều 10

Hạng	Cosine	Node
4	0,3666	Quy chế 2021 – Điều 7
5	0,3202	QĐ 3183/2025 – sửa đổi Điều 7
"A thì được bao nhiêu điểm hệ 10"		
1	0,3843	QĐ 3183/2025 – sửa đổi Điều 9
2	0,3605	QĐ 3183/2025 – sửa đổi Điều 10
3	0,3373	Quy chế 2021 – Điều 10
4	0,3334	QĐ 3183/2025 – mục sửa đổi Điều 9 thứ hai (QĐ có hai mục riêng: điểm a và điểm d khoản 3)
5	0,2823	Quy chế 2021 – Điều 9
"Điều kiện để được học cùng lúc hai chương trình?"		
1	0,6192	QĐ 810 – Điều 3
2	0,5895	QĐ 810 – Điều 2
3	0,5641	Quyết định ban hành QĐ 810 – Điều 2
4	0,5603	Quy chế 2021 – Điều 17
5	0,5502	QĐ 810 – Điều 4

Thảo luận. Điểm cosine thấp (0,3–0,6) và các hạng sát nhau (hạng 1 và 2 của câu đầu chỉ chênh 0,004). Điều này có hai hệ quả: (1) không thể đặt một ngưỡng cố định để loại Node "không liên quan" mà không loại nhầm Node đúng — lý do similarity_cutoff mặc định tắt; (2) thứ tự giữa Điều gốc và mục sửa đổi của cùng một Điều gần như ngẫu nhiên, nên cả hai cần cùng nằm trong ngữ cảnh. Câu về điểm A cho thấy cả bảng hệ 10 (Điều 9) lẫn bảng hệ 4 (Điều 10) đều lọt vào top-5 — nguồn gốc của lỗi nhầm thang điểm ở Mục 4.5.2.

Trên bộ 31 câu hỏi về Quy chế 2021 đo lại ngày 17/10/2026 trên corpus 118 Node, cấu hình vector cho R@1 = 80,6%, R@3 = 96,8%, MRR = 0,882. Phân bố văn bản của Node top-1:

Đoạn mã 4.3: Văn bản của Node đứng đầu, bộ 01, cấu hình vector.

```
Quy chế đào tạo trình độ đại học SGU 2021 : 23 câu
QĐ 3183/2025 sửa đổi Quy chế đào tạo SGU :  4 câu
QĐ 810 về học cùng lúc hai chương trình   :  3 câu
QĐ 2375/2022 về chuyển chương trình       :  1 câu
```

Bốn câu còn lại cho thấy tác dụng phụ của việc mở rộng corpus: văn bản khác chủ đề nhưng dùng từ ngữ tương tự chen lên đầu. Trên bộ 02 (10 câu về nội dung sửa đổi) 9/10 câu lấy QĐ 3183 ở top-1 (câu còn lại bị TB 2567 về quy đổi tiếng Anh chen lên), và câu hỏi về *khóa tuyển sinh áp dụng* thang điểm mới trượt khỏi top-3 vì đáp án nằm ở Điều 2 (điều khoản hiệu lực) trong khi truy hồi trả về các mục sửa đổi Điều 9, 10 — chúng khớp chữ "B+" nhiều hơn.

Kết quả đầy đủ của tám cấu hình nằm ở Chương 8.

4.5 Tổng hợp câu trả lời (Response synthesis)

4.5.1 Cơ chế

Module src/query_engine.py ghép retriever, postprocessor và LLM thành RetrieverQueryEngine:

- **LLM**: Ollama(model=..., thinking=cfg.thinking, context_window=8192) [10], [11]. Mặc định tắt chế độ suy nghĩ của Qwen3 để câu trả lời ngắn; có thể bật bằng biến môi trường RAG_THINKING=1 (trường thinking trong RAGConfig, được ghi vào mo_ta() để kết quả đánh giá biết đã đo ở chế độ nào). Bật chế độ này giúp tra bảng khoảng điểm cẩn thận hơn nhưng chậm khoảng bốn lần.
- **Tất định**: temperature=0 và seed=42 truyền xuống Ollama. Không đặt hai tham số này thì Ollama lấy mẫu ngẫu nhiên: nhật ký hỏi đáp trước khi sửa ghi cùng một câu hỏi, lần đầu hệ thống từ chối, lần sau trả lời "Không".
- **Prompt tiếng Việt riêng** thay prompt mặc định: nêu rõ ngữ cảnh là trích đoạn quy chế SGU, yêu cầu *chỉ* dựa vào trích đoạn, và bổ sung ba ràng buộc nhắm đúng các lỗi đã ghi trong nhật ký: (1) *trích nguyên văn câu quy định trước khi kết luận*, (2) *không tự quy đổi thang điểm hay đổi đơn vị, cần tra bảng thì chép đúng hàng của bảng*, (3) nói "không tìm thấy" khi trích đoạn không có thông tin — cho mô hình một lối ra hợp lệ thay vì phải đoán.
- **Trích dẫn nguồn**: format_sources() đọc metadata của các Node nguồn và in ra "Nguồn: Điều X, Điều Y". Nguồn lấy từ metadata chứ không để LLM tự ghi, nên luôn khớp với những gì đã đưa vào prompt.
- **Streaming**: câu trả lời in dần từng token qua response.response_gen.

4.5.2 Sửa lỗi nhầm thang điểm hệ 10 và hệ 4

Nhật ký câu hỏi của giao diện Gradio (eval/results/questions_log.jsonl, mỗi câu hỏi một dòng JSON gồm câu hỏi, cấu hình, các Node nguồn và câu trả lời) ghi lại một lỗi lặp lại: với câu "A thì được bao nhiêu điểm hệ 10", hệ thống trả lời *"A được quy đổi thành 4,0 điểm hệ 10"* ở cả hai lần hỏi (top-10 và top-3). Đáp án đúng là A ứng với khoảng 8,5–10 trên thang điểm 10 (khoản 3 Điều 9); con số 4,0 là giá trị quy đổi sang thang điểm 4 (khoản 2 Điều 10).

Truy hồi không sai: cả Điều 9 và Điều 10 (bản gốc lẫn bản sửa đổi của QĐ 3183) đều nằm trong ngữ cảnh. Nguyên nhân nằm ở dữ liệu: mục sửa đổi của QĐ 3183 chỉ chứa *bảng* ("A: từ 8,5 đến 10.0", "A quy đổi thành 4,0") mà không nhắc tên thang điểm, nên khi hai bảng đứng cạnh nhau trong ngữ cảnh, LLM không có căn cứ để biết bảng nào là "hệ 10" và bảng nào là "hệ 4". Cách sửa là thêm vào QA_PROMPT một đoạn chú giải miền (domain note) ngắn:

Đoạn mã 4.4: Chú giải thang điểm thêm vào QA_PROMPT (src/query_engine.py).

```
Lưu ý về thang điểm: điểm thành phần và điểm học phần chấm theo thang điểm 10 ("hệ 10"),
rồi xếp loại thành điểm chữ (A, B+, B, ...) theo khoảng điểm ở khoản 3 Điều 9. Điểm chữ được
quy đổi sang thang điểm 4 ("hệ 4") chỉ để tính điểm trung bình, theo khoản 2 Điều 10.
Hỏi về hệ 10 hoặc khoảng điểm thì dùng bảng Điều 9; hỏi về hệ 4 hoặc quy đổi để tính điểm
trung bình thì dùng bảng Điều 10. Khi tra khoảng điểm, so sánh con số với cả cận dưới và
cận trên của từng hàng.
```

Đây là giải pháp tạm và là một đánh đổi có ý thức: prompt chứa kiến thức riêng của quy chế SGU, nên không dùng lại được cho văn bản khác. Nhưng thông tin "bảng nào thuộc thang nào" vốn có trong văn bản gốc (câu dẫn của khoản 3 Điều 9 và khoản 2 Điều 10) và chỉ bị mất khi cắt Node theo mục sửa đổi; cách sửa triệt để hơn là gắn câu dẫn đó vào metadata của mục sửa đổi. Một phần kiến thức miền đã được chuyển từ prompt sang dữ liệu: khóa áp dụng của từng mục sửa đổi (ap_dung_khoa_tu_theo_dieu, mục sửa Điều 9 và 10 chỉ áp dụng từ khóa 2025) và quan hệ sửa đổi giữa các văn bản nay nằm trong front matter, và bước lọc phiên bản dùng chúng mà không cần prompt (Mục 8.2). Câu dẫn "thang điểm 10"/"thang điểm 4" chưa được gắn vào metadata nên chú giải vẫn giữ trong prompt. Hiệu quả của chú giải chưa được đo; phép so sánh có và không có chú giải sẽ chạy cùng đợt sinh câu trả lời bằng eval/tra_loi_dev.py.

4.5.3 Hạn chế của cấu hình này

RetrieverQueryEngine chỉ gọi LLM *một lượt* trên ngữ cảnh đã truy hồi: nó không nhớ lịch sử hội thoại, nên câu hỏi nối tiếp kiểu "còn kỹ sư?" không đủ thông tin để truy hồi. Muốn sửa phải đổi sang chat engine có viết lại câu hỏi theo lịch sử (ví dụ CondensePlusContextChatEngine của LlamaIndex) — việc này được nêu ở Chương 12 như hướng phát triển, và đã có sẵn lịch sử chat trong giao diện Gradio để dùng.

4.6 Đánh giá thực nghiệm

4.6.1 Độ đo

Do chất lượng RAG phụ thuộc đầu tiên vào việc tìm đúng đoạn, đồ án đánh giá riêng bước truy hồi trước khi đánh giá câu trả lời. Bộ câu hỏi chia hai nhóm. Tập phát triển (dev) được dùng trong quá trình chỉnh hệ thống, gồm 91 câu: bộ 01 (31 câu), bộ 02 (10 câu) và ba đợt dev 1–3 (20, 15 và 15 câu về các văn bản nạp thêm). Tập kiểm chứng (test) gồm 40 câu trong phạm vi và 20 câu ngoài phạm vi, do hai thành viên không chỉnh pipeline viết và gán nhãn độc lập (trùng 40/40 nhãn, $\kappa = 1{,}00$), được đóng băng trước lần chạy duy nhất. Mỗi câu có hai trường question và dieu_dung (Điều hoặc văn bản chứa đáp án, có thể là danh sách nhiều nhãn cùng đúng). Hai bộ dev đầu tiên:

- eval/test_questions_01.json: 31 câu về Quy chế đào tạo 2021;
- eval/test_questions_02.json: 10 câu về các nội dung được *sửa đổi* bởi Quyết định 3183/2025 (rút học phần, số học kỳ, thang điểm chữ B+, C+, D+…).

Nhờ có nhãn, có thể dùng hai độ đo phổ biến. Gọi $n_{i,j}$ là Node đứng thứ j trong danh sách trả về cho câu hỏi thứ i, N là số câu hỏi:

- **Recall@k** (còn gọi là hit rate@k): tỷ lệ câu hỏi mà trong k Node đầu tiên có ít nhất một Node thuộc Điều đúng. Vì mỗi câu chỉ có một Điều đúng, hai cách gọi này trùng nhau:

$$Recall@k = \frac{1}{N}\sum_{i=1}^{N} 1\left[\exists\, j \le k : dieu(n_{i,j}) = dieu_dung_i\right].$$

- **MRR** (Mean Reciprocal Rank): trung bình nghịch đảo thứ hạng của Node đúng đầu tiên (bằng 0 nếu không tìm thấy):

$$MRR = \frac{1}{N}\sum_{i=1}^{N}\frac{1}{rank_i}.$$

Recall@k cho biết Điều đúng *có lọt vào* ngữ cảnh đưa cho LLM hay không; MRR cho biết nó *đứng cao* tới đâu. Đây là các độ đo chuẩn của bài toán truy hồi [15].

Cụ thể, bộ 01 (31 câu về Quy chế 2021) và bộ 02 (10 câu về nội dung sửa đổi bởi Quyết định 3183/2025). Vì corpus có nhiều văn bản trùng số Điều, mỗi bộ có luật riêng xác định Node nào là "đúng", chấm theo cặp (số hiệu văn bản, Điều) lấy từ front matter. Bộ 01 nhận Node phần chính của Quy chế 2021 (258/QĐ-ĐHSG) có Điều thuộc nhãn, hoặc mục QĐ 3183 sửa đổi chính Điều đó. Bộ 02 chỉ nhận Node của QĐ 3183 (mục sửa đổi Điều thuộc nhãn hoặc Điều của chính QĐ). Các đợt dev nhận Node của văn bản ghi trong nhãn, và đúng Điều nếu nhãn có Điều. Vì luật bộ 01 rộng hơn bộ 02, R@1 của hai bộ không so trực tiếp với nhau. Luật chấm có kiểm thử riêng trong tests/test_luat_cham.py.

Ba độ đo, khoảng tin cậy bootstrap và kiểm định McNemar được cài trong eval/metrics.py:

- **Recall@k**: tỷ lệ câu có Node đúng trong k hạng đầu.
- **MRR**: trung bình nghịch đảo hạng của Node đúng đầu tiên.
- **nDCG@k**: như Recall nhưng tính cả vị trí, không chỉ "có hay không".
- **Khoảng tin cậy bootstrap** (2 000 lần lấy mẫu lại) cho mọi chỉ số — bắt buộc khi bộ câu hỏi chỉ có 10–31 câu.
- **Kiểm định McNemar chính xác**: so hai cấu hình trên *cùng* bộ câu hỏi, trả lời câu hỏi "chênh lệch này có thể chỉ do may mắn không".

Ngoài ra eval/evaluate.py --kiem-chung chấm lại cùng bộ câu hỏi bằng RetrieverEvaluator của LlamaIndex (các metric hit_rate, mrr) để đối chiếu cách đo tự viết với cách đo của thư viện.

4.6.2 Kết quả

Môi trường đo. Số liệu truy hồi được sinh lại ngày 17/10/2026 bằng eval/evaluate.py (make eval) trên GPU RTX A2000 4 GB, embedding và reranker ở fp16. Baseline ở Bảng 4.4 được đo ngày 08/10 trên corpus 54 Node. Mọi kết quả được ghi kèm

cấu hình và thời điểm đo trong eval/results/*.json; bảng tổng hợp ở eval/results/logs/2026-10-17_chot-cau-hinh/04_tong_hop.md.

Bảng 4.3: Truy hồi bằng vector top-10 (không rerank, không lọc phiên bản), profile server, 118 Node, đo ngày 17/10/2026 (eval/results/truy-hoi.md). Cột test chưa có vì tập kiểm chứng chỉ chạy một lần sau khi đóng băng.

Bộ	Cấu hình	R@1 (CI 95%)	R@3	R@10	MRR	nDCG@3
01 (31 câu)	vector	81% [68–94]	97%	100%	0,882	0,900
02 (10 câu)	vector	90% [70–100]	90%	90%	0,900	0,900

Bảng 4.4: Baseline (sinh lại bằng make baseline). "Không xếp hạng" trả về Node theo thứ tự docstore — tức mốc sàn khi không có bước xếp hạng nào.

Bộ	Baseline	R@1 (CI 95%)	R@3	R@10	MRR	ms/câu
01 (31 câu)	BM25 (tìm từ khóa)	58% [42–74]	84%	97%	0,727	0,2
	không xếp hạng	0%	0%	0%	0,053	0,05
02 (10 câu)	BM25 (tìm từ khóa)	40% [10–70]	60%	90%	0,537	0,3
	không xếp hạng	0%	0%	0%	0,020	0,05

Mô hình embedding là yếu tố quyết định, và thiết kế "cắt theo Điều" là điều kiện để nó phát huy. Trên bộ 01 (đo cùng ngày 08/10 với baseline, corpus 54 Node), vector đạt R@1 = 81% và R@3 = 100%, trong khi BM25 — đúng kiểu tìm kiếm từ khóa mà sinh viên đang dùng trên file PDF — chỉ đạt R@1 = 58% và R@3 = 84%. Khoảng cách 23 điểm phần trăm ở R@1 là kết quả có ý nghĩa thực tiễn lớn nhất của đồ án: nó định lượng được cái mà phần mở đầu chỉ nói bằng lời, rằng "cấm thi" không tìm được "không đủ điều kiện dự thi" bằng từ khóa.

Kiểm chứng chéo bằng thư viện. Chạy eval/evaluate.py --kiem-chung ngày 17/10, mrr do RetrieverEvaluator của LlamaIndex tính trùng với mrr tự viết ở cả năm bộ dev: 0,882 (bộ 01), 0,900 (bộ 02), 0,963, 0,813 và 0,900 (dev đợt 1–3). hit_rate bằng 1,000 ở bốn bộ và 0,900 ở bộ 02, đúng bằng Recall@10 tương ứng. Hai cách đo độc lập cho cùng kết quả là một kiểm tra tính đúng đắn của bộ đánh giá.

Ca trượt còn lại và nguyên nhân. Trên bộ 02, câu duy nhất nằm ngoài top-3 ở cấu hình vector là "Thang điểm chữ bổ sung (B+, C+, D+) và quy đổi điểm mới áp dụng cho khóa tuyển sinh nào?" — đáp án nằm ở Điều 2 (điều khoản hiệu lực) của QĐ 3183, nhưng truy hồi trả về các mục sửa đổi Điều 9 và 10 vì chúng chứa ký hiệu "B+" nhiều lần. Đây là dạng lỗi "khớp bề mặt thắng khớp ngữ nghĩa" đặc trưng của kho nhỏ, và là lý do nên bổ sung metadata hiệu lực. Câu này vẫn trượt top-1 ở cấu hình chốt (Mục 8.3).

4.6.3 Câu hỏi tự kiểm

"Cái gì trong đồ án này là LlamaIndex mà một dự án không dùng LlamaIndex sẽ không làm giống như vậy?" Ba điều: (1) Node mang metadata và *tự quyết định* trường nào vào embedding, trường nào vào prompt (excluded_embed_metadata_keys / excluded_llm_metadata_keys); (2) truy hồi, hậu xử lý và tổng hợp là ba interface rời, nên BM25 tự viết và reranker cắm vào mà không đổi phần còn lại; (3) chỉ mục lưu/nạp bằng StorageContext cùng docstore, nên Node truy hồi được luôn mang theo metadata để trích nguồn. Phần dữ liệu (OCR, Markdown hai tầng) thì *không* phải LlamaIndex — dự án nào cũng phải làm.

4.6.4 Tập kiểm chứng chống quá khớp

Bộ 01, bộ 02 và các đợt dev đã được dùng để chỉnh loader, prompt và chọn cấu hình, nên chỉ số trên chúng có thể lạc quan. Vì vậy nhóm lập một tập kiểm chứng riêng do hai thành viên không chỉnh pipeline thực hiện, theo quy tắc độc lập: không mở bộ dev và kết quả đánh giá, không chạy hệ thống, chỉ đọc văn bản trong data/processed và PDF gốc. Bảng 4.7 tóm tắt các bước.

Bảng 4.7: Các bước lập tập kiểm chứng (tuần 6).

Bước	**Người làm**	**Sản phẩm**
K1: viết câu	Võ Thị Như Ý	40 câu trong phạm vi (32 về Quy chế 2021, 8 về QĐ 3183), chia 6 loại: hỏi thẳng 7, diễn đạt khác 8, khẩu ngữ 5, tình huống 6, tra bảng 6, nội dung sửa đổi 8; mỗi câu có Điều đúng và câu trích nguyên văn
K2: gán nhãn độc lập	Trần Hà Thanh Thùy	Nhãn Điều cho 40 câu khi chỉ

		thấy câu hỏi; 20 câu ngoài phạm vi (6 ngoài hẳn, 8 sát biên, 6 câu bẫy) kèm hành vi mong đợi; nhãn phạm vi cho 56 câu của phiên hỏi đáp cũ
K3: đối chiếu nhãn	Tính bằng script; họp chốt và đóng băng do hai người trên thực hiện	Trùng 40/40 nhãn, Cohen's κ = 1,00; 4 câu gần như trùng ý với câu dev cần viết lại hoặc ghi lý do giữ

Hai người gán nhãn ở mức Điều và mỗi câu được viết để chỉ một Điều trả lời được, nên việc trùng 40/40 nhãn là hợp lý; con số này không nói gì về độ khó ở mức khoản. Tập kiểm chứng chưa được chạy. Thứ tự đã thống nhất là: chốt cấu hình trên tập dev (Chương 8), đóng băng tập kiểm chứng bằng mã băm sha256 của các file JSON, rồi chạy đúng một lần. Số liệu đó sẽ điền vào cột test và không được dùng để chỉnh hệ thống (chi tiết ở eval/results/do_dong_thuan.md).

CHƯƠNG 5. CUSTOM NODE PARSER THEO CẤU TRÚC VĂN BẢN PHÁP QUY

5.1 Vì sao chọn

Văn bản quy chế chia theo Chương — Điều — khoản, và văn bản sửa đổi (QĐ 3183) chia theo "mục sửa đổi Điều *N*". Câu trả lời và trích dẫn đều theo Điều, nên Node phải trùng ranh giới Điều. Bộ cắt mặc định của LlamaIndex (SentenceSplitter) không biết ranh giới này.

Cách làm ở tuần 3 (src/ingestion.py) là OCR từng trang rồi đóng gói *mỗi trang thành một Document*. Cách đó có hai nhược điểm đã đo được: (1) một Điều nằm vắt qua hai trang bị chia đôi; (2) phần "Quyết định ban hành" ở trang đầu sinh ra các Node trùng số Điều với phần Quy chế (Điều 1–3 xuất hiện hai lần), làm hỏng phép chấm truy hồi theo số Điều. Hai module src/ingestion.py và src/node_parser.py được giữ lại trong repository để đối chiếu nhưng **không còn nằm trong pipeline**; dữ liệu vào Node ngày nay đi qua tầng 2 nêu trên.

5.2 Cơ chế

Cơ chế tách theo heading được mô tả ở Mục 4.2. Phần dưới là hai quyết định về metadata — nơi custom parser khác hẳn parser mặc định.

Loại metadata rỗng. Loader chỉ gắn khóa có giá trị. Nếu gắn cho *mọi* Node bốn khóa van_ban, chuong, dieu, sua_doi_dieu, kể cả khi giá trị rỗng, thì vì LlamaIndex ghép metadata vào văn bản trước khi embedding, mỗi vector đều chứa các dòng vô nghĩa kiểu chuong:, dieu:, làm nhiễu vector của mọi Node.

Chỉ một trường được embedding. Loader sinh thêm khóa nguon_trich và chỉ trường này được đưa vào embedding; các khóa kỹ thuật còn lại bị loại bằng excluded_embed_metadata_keys. Văn bản đưa vào bộ mã hóa vì thế có dạng:

Đoạn mã 5.1: Văn bản đưa vào embedding của một Node (in bằng metadata_mode="embed").

```
nguon_trich: Điều 9 — Quy chế đào tạo trình độ đại học SGU 2021

Điều 9. Đánh giá và tính điểm học phần
]. Đối với mỗi học phần, người học được đánh giá qua tối thiểu hai điểm thành phần; các điểm thành phần được
```

đánh giá theo thang điểm 10, Hình thức đánh giá, phương pháp đánh giá và trọng số của mỗi điểm thành phần được quy định tại đề cương chi tiết của mỗi học phần. … (Node dài 2 332 ký tự, chỉ trích phần đầu)

Nếu để metadata mặc định, cùng Node này sẽ được embed kèm chuỗi van_ban: ...\nchuong: \ndieu: 9\nsua_doi_dieu: — vừa nhiễu vừa chứa khóa tiếng Anh không dấu. Ký tự "]." ở đầu khoản 1 và "chỉ tiết" là lỗi OCR còn trong tầng 2, giữ nguyên để thấy đúng văn bản mô hình nhận được. Chỉ số nguon_trich cũng chính là nhãn node_label() dùng để in nguồn, nên phần hiển thị và phần embedding không còn lệch nhau. Đây là ví dụ cho thấy một quyết định nhỏ ở tầng Node ảnh hưởng trực tiếp tới chất lượng vector.

5.2.1 Mã nguồn

Đoạn mã 5.2: Gắn metadata đã lọc rỗng và chọn trường đưa vào embedding (src/processed_loader.py).

```
def _gan_metadata(node: TextNode, meta: dict[str, str | int]) -> TextNode:
    """Gắn metadata đã lọc rỗng + `nguon_trich`, và đặt đúng trường nào được embed / vào prompt."""
    sach = {k: v for k, v in meta.items() if str(v).strip()}
    sach["nguon_trich"] = nguon_trich(sach)
    node.metadata = sach
    node.excluded_embed_metadata_keys = [k for k in sach if k != "nguon_trich"]
    node.excluded_llm_metadata_keys = [k for k in KHONG_LLM if k in sach]
    return node
```

5.3 Đã làm gì

- Viết src/processed_loader.py thay cho node_parser.py (regex DIEU_PATTERN trên văn bản thô) — nay dựa vào heading do clean_md.py dựng.
- Tách riêng phần "Quyết định ban hành" (phan="ban_hanh") để Điều 1–3 của quyết định không trùng nhãn với Điều 1–3 của quy chế.
- Gắn LOADER_VERSION để chỉ mục tự build lại khi logic cắt đổi (Chương 6).

5.4 Số liệu

118 Node từ 20 văn bản (Mục 4.1.3); chỉ mục và loader khớp 118/118 (Mục 3.4). Trên corpus 54 Node, truy hồi vector đạt R@1 81% trên bộ 01, cao hơn BM25 23 điểm phần trăm (Mục 4.6). Độ dài Node ở Bảng 4.6.

Chưa triển khai. Bảng so sánh cách chunk (B1) — xem Mục 4.2.

5.5 Kiểm thử

tests/test_processed_loader.py khóa lại đúng những hành vi trên: số Node và nhãn Điều, việc loại metadata rỗng, việc chỉ nguon_trich được embedding, và giá trị của nguon_trich cho ba trường hợp (Điều thường, mục sửa đổi, phần ban hành). tests/test_corpus.py kiểm tra trên corpus thật: mọi tiền tố trong danh sách văn bản khớp đúng một file, và mọi Node đều có nguon_trich. Chạy make test: 54 kiểm thử, trong đó 10 kiểm thử cần Ollama tự bỏ qua; phần còn lại không cần GPU. tests/test_phien_ban.py khóa lại bước lọc phiên bản (Mục 8.2), tests/test_luat_cham.py khóa lại luật chấm (Mục 4.6).

5.6 Hạn chế đã biết

- Cách cắt phụ thuộc clean_md.py dựng heading đúng. Nếu một văn bản bị OCR làm mất dòng "Điều *N*." thì Điều đó bị gộp vào Điều trước hoặc vào Node Chương.
- Ngưỡng MAX_TU đếm theo từ còn SentenceSplitter đếm theo token tiktoken (Mục 4.2.1); nên đổi ngưỡng sang đếm bằng chính tokenizer của mô hình embedding.
- 10 văn bản không chia Điều cho mỗi văn bản đúng một Node, chưa cắt theo khoản.

5.7 Dùng ở đâu trong đồ án

Mọi bước sau đều đi trên Node này: chỉ mục, truy hồi, nhãn nguồn node_label(), và luật chấm đúng/sai theo (văn bản, Điều) của bộ đánh giá.

CHƯƠNG 6. PERSISTENT INDEX VÀ QUẢN LÝ CẤU HÌNH

6.1 Vì sao chọn

Embedding toàn bộ corpus mất khoảng 4 giây trên GPU 4 GB và hơn một phút trên CPU; dựng lại mỗi lần hỏi là không chấp nhận được. Ngược lại, dùng lại chỉ mục cũ sau khi đã đổi mô hình embedding hay cách cắt Node thì cho kết quả sai mà không báo lỗi. Cần lưu chỉ mục *và* biết khi nào nó hết hợp lệ.

6.2 Cơ chế

Hàm build_or_load_index() gọi storage_context.persist() sau khi dựng chỉ mục; ở các lần sau, nó tạo StorageContext từ thư mục đã lưu rồi gọi load_index_from_storage(). Thư mục lưu gồm docstore, vector store và index store dạng JSON.

6.2.1 Fingerprint: chống dùng lại chỉ mục cũ

Nếu chỉ kiểm tra tên mô hình embedding và sự tồn tại của docstore.json, thì sửa cách cắt Node mà quên --rebuild sẽ âm thầm dùng chỉ mục cũ. Vì vậy build_or_load_index() lưu corpus.json cạnh chỉ mục, gồm: model embedding, thư mục corpus, **phiên bản loader cùng các ngưỡng cắt**, và sha256 + tên hiển thị của từng file. Chỉ cần một trong các thứ đó đổi là chỉ mục tự build lại.

Đoạn mã 6.1: Fingerprint của chỉ mục (src/index_builder.py).

```
return {
    "embed_model": cfg.embed_model_name,
    "corpus_dir": cfg.corpus_dir,
            "loader": {"phien_ban": LOADER_VERSION, "max_tu": MAX_TU, "min_tu_noi_dung":
MIN_TU_NOI_DUNG},
    "files": files,
  }
```

persist_dir được suy ra thành storage/<profile>/<đường dẫn corpus>, nên hai profile có số chiều vector khác nhau không ghi đè lên nhau. Vì số chiều vector gắn với mô hình, đổi mô hình embedding vẫn phải build lại chỉ mục — corpus.json cạnh chỉ mục sẽ tự phát hiện.

6.3 Số liệu

Bảng 6.1: Thời gian dựng mới và nạp lại chỉ mục.

Thao tác (profile server, 118 Node, GPU fp16, đo 17/10)	**Thời gian**
Dựng mới (embedding toàn bộ Node; ngày 09/10 trên CPU, 54 Node mất 77 giây)	khoảng 4 giây
Nạp lại chỉ mục từ đĩa (không tính thời gian nạp mô hình embedding)	khoảng 0,5 giây

6.4 Dùng ở đâu trong đồ án

Mọi điểm vào (pipeline.py, app/app.py, eval/*.py, notebook) đều gọi build_or_load_index(), nên không có đường nào dùng nhầm chỉ mục cũ.

CHƯƠNG 7. TRÍCH DẪN NGUỒN VÀ TỪ CHỐI CÂU HỎI NGOÀI CORPUS

7.1 Vì sao chọn

Câu trả lời về quy chế chỉ có giá trị khi người đọc kiểm tra lại được. Và với câu hỏi mà quy chế không nói tới, "không tìm thấy" là câu trả lời đúng — bịa ra một con số là lỗi nặng nhất.

7.2 Cơ chế

Trích dẫn. format_sources() đọc metadata của các Node nguồn qua node_label() và in "Nguồn: <văn bản> – Điều N". Nguồn lấy từ metadata chứ không để LLM tự ghi, nên luôn khớp với những gì đã vào prompt. Prompt còn bắt LLM trích nguyên văn câu quy định kèm tên văn bản và số Điều trước khi kết luận.

Từ chối. Hai lớp: (1) prompt cho LLM một lối ra hợp lệ — "nếu trích đoạn không có thông tin, hãy nói không tìm thấy"; (2) SimilarityPostprocessor loại Node dưới ngưỡng (similarity_cutoff), đã cài nhưng mặc định tắt vì điểm cosine thấp và sát nhau (Mục 4.4).

7.3 Đã làm gì

- Nhãn nguồn thống nhất với văn bản embedding qua trường nguon_trich (Chương 5).
- Bộ eval/cau_hoi_danh_gia.json: 13 câu, mỗi câu có pham_vi (10 "trong", 3 "ngoài") để tách từ chối đúng khỏi từ chối sai. Tập kiểm chứng có thêm 20 câu ngoài phạm vi (eval/test_kiem_chung_ngoai.json: 6 câu ngoài hẳn, 8 câu sát biên, 6 câu bẫy), mỗi câu ghi hành vi mong đợi.

7.4 Số liệu

Trên 56 câu hỏi tự đặt (Chương 10), tỷ lệ trả lời "không tìm thấy" là 52% ở phiên đầu và 41% ở phiên chạy lại ngày 08/10. Chấm tay cho thấy cả hai loại: từ chối đúng ("điểm rèn luyện là gì", "bao nhiêu tiền một tín chỉ") và từ chối sai ("còn kỹ sư", "rớt một môn có được xuất sắc không").

Chưa triển khai. Tỷ lệ từ chối đúng trên 20 câu ngoài phạm vi sẽ được đo trong lần chạy tập kiểm chứng duy nhất. Trích dẫn mới tới văn bản và Điều, chưa tới

trang hay khoản: số trang có ở tầng 1 (dòng "## Trang N" do extract_to_md.py ghi) nhưng bị clean_md.py bỏ khi làm sạch. Hướng sửa là giữ số trang dưới dạng chú thích ẩn rồi đưa vào metadata của Node, để nhãn nguồn in được dạng "Điều 9, tr. 6–7". Vì vậy yêu cầu metadata tên file, trang, chương mới đạt một phần.

7.5 Dùng ở đâu trong đồ án

Mọi câu trả lời ở CLI, giao diện Gradio và nhật ký đánh giá đều kèm dòng "Nguồn".

CHƯƠNG 8. KỸ THUẬT NÂNG CAO ĐÃ CHỌN: HYBRID SEARCH VÀ XẾP HẠNG LẠI

8.1 Vì sao chọn

Câu hỏi về quy chế chứa ký hiệu và con số mà embedding nắm kém ("B+", "Điều 10", "150 tín chỉ"), và bi-encoder chỉ so khớp ở mức ý chính. Hai kỹ thuật thường được khuyến nghị cho đúng hai điểm yếu này là trộn BM25 (hybrid search) và xếp hạng lại bằng cross-encoder. Đồ án cài cả hai và *đo* xem chúng có giúp trên corpus của mình không.

8.2 Cơ chế

Module src/retriever.py tổ chức truy hồi thành ba tầng độc lập, đúng theo cách LlamaIndex tách *retriever* khỏi *node postprocessor*:

1. **Vector top-*k* (mặc định)**: index.as_retriever(similarity_top_k=10). Câu hỏi được embedding bằng đúng mô hình đã dùng khi lập chỉ mục, rồi so cosine với toàn bộ Node (SimpleVectorStore tìm vét cạn, phù hợp với kho vài chục đến vài trăm Node).
2. **Hybrid BM25 + vector (tuỳ chọn, use_hybrid)**: câu hỏi về quy chế chứa nhiều ký hiệu và con số mà embedding nắm kém ("B+", "Điều 10", "150 tín chỉ", "QĐ 3183"). Đồ án cài BM25 Okapi [7] thuần Python trong src/bm25.py — gói BM25Retriever chính thức nằm ở một package riêng chứ không có trong llama-index-core, nên tự viết vừa nhẹ vừa cho phép kiểm soát cách tách từ. Hai nguồn được trộn bằng QueryFusionRetriever ở chế độ reciprocal_rerank (RRF), với num_queries=1 để không gọi LLM sinh truy vấn phụ và giữ phép đo tất định.
3. **Postprocessor**: theo thứ tự: ngưỡng tương đồng SimilarityPostprocessor (tuỳ chọn); lọc phiên bản LocPhienBan; xếp hạng lại bằng cross-encoder [16] (SentenceTransformerRerank, đọc 2 048 token mỗi cặp); cuối cùng UuTienSuaDoi đưa bản sửa đổi lên trước rồi cắt top-3.

Tách từ cho BM25. tach_tu() bỏ dấu tiếng Việt trước khi chấm điểm (Điều → dieu). Nhờ vậy câu hỏi gõ thiếu dấu hoặc lỗi OCR vẫn khớp văn bản có dấu — đúng loại lỗi đã gặp trong nhật ký hỏi đáp. Đánh đổi là mất phân biệt các cặp chỉ khác dấu, hiếm gặp trong văn bản quy chế. tests/test_bm25.py khóa lại hành vi này bằng ba ca: khớp theo ký hiệu

"B+", khớp khi câu hỏi không dấu, và trả về rỗng khi không có từ nào khớp (tránh đưa Node nhiễu vào prompt).

Một chi tiết dễ sai. Điểm sau khi trộn RRF không cùng thang với cosine, nên nếu đặt similarity_cutoff khi đang bật hybrid thì mọi Node đều bị loại. Hàm get_postprocessors() vì thế chủ động bỏ qua ngưỡng khi use_hybrid=True và ghi log cảnh báo, thay vì trả về danh sách rỗng một cách im lặng.

Chạy trên GPU 4 GB và trên máy từ xa. Reranker nạp ở fp16 (Mục 3.1.4). Khi đặt MODEL_SERVER_URL, embedding và reranker được gọi qua HTTP tới src/model_server.py (RemoteEmbedding, RemoteRerank trong src/remote_models.py); client kiểm tra server đang chạy đúng mô hình của profile để vector không lệch với chỉ mục.

Lọc phiên bản (src/phien_ban.py). Bước này dùng metadata hiệu lực trong front matter (Mục 4.1.2) và gồm hai postprocessor. LocPhienBan đặt trước reranker: bỏ Node của văn bản hết hiệu lực hoặc bị thay thế; nếu câu hỏi nêu khóa tuyển sinh ("khóa 2020", "K22") thì bỏ Node không áp dụng cho khóa đó, nhưng chỉ khi cùng chuỗi văn bản còn Node áp dụng được, để không mất hết căn cứ; khi lấy được một Điều gốc có mục sửa đổi thì kéo kèm mục sửa đổi đó dù vector không lấy ra. UuTienSuaDoi đặt sau reranker: đưa văn bản sửa đổi lên ngay trước văn bản bị sửa rồi mới cắt top-3, vì vậy reranker giữ đủ Node. Lý do của ngưỡng điểm ở Mục 8.3; tests/test_phien_ban.py khóa lại các trường hợp này trên dữ liệu mô phỏng corpus thật.

Reranker đọc 2 048 token. SentenceTransformerRerank tạo CrossEncoder với max_length = 512 cố định và không cho truyền tham số này. Hàm dat_do_dai_toi_da() trong src/retriever.py đặt lại thành 2 048 sau khi nạp (từ sentence-transformers 5, thuộc tính đổi tên thành max_seq_length). Model server làm tương tự qua biến môi trường RERANK_MAX_LENGTH, và client từ chối chạy nếu server báo độ dài khác cấu hình, để điểm rerank không âm thầm sai.

8.3 Số liệu

Bảng 8.1: Số câu đúng ở top-1 theo cấu hình trên 91 câu dev, profile server, 118 Node, GPU fp16, đo ngày 17/10/2026 (eval/results/truy-hoi.md). Rerank đọc 2 048 token, giữ 3 Node; luật chấm ở Mục 4.6.1. Cột nDCG@1 được bỏ vì với nhãn nhị phân nDCG@1 trùng R@1; ms/câu là trung vị thời gian truy hồi.

Cấu hình	**Bộ 01 (31)**	**Bộ 02 (10)**	**Dev đợt 1 (20)**	**Dev đợt 2 (15)**	**Dev đợt 3 (15)**	**R@1 gộp (91)**	**MRR gộp**	**ms/câu**
vector	25	9	19	10	12	82,4%	0,893	47
rerank	27	5	18	14	12	83,5%	0,903	779
lọc phiên bản	25	9	18	11	14	84,6%	0,897	59
rerank + lọc phiên bản (chốt)	27	7	20	15	14	91,2%	0,947	887
hybrid	25	6	18	11	10	76,9%	0,857	45
hybrid + rerank	27	5	18	14	12	83,5%	0,903	854
hybrid + rerank + lọc phiên bản	27	7	20	15	14	91,2%	0,947	861

Rerank chỉ có ích khi đọc đủ Node và đi cùng bước lọc phiên bản. Trên corpus 54 Node, rerank từng làm bộ 02 tụt từ 90% xuống 60%, nên báo cáo tuần 5 kết luận tắt rerank. Kiểm tra lại cho thấy SentenceTransformerRerank của LlamaIndex tạo CrossEncoder với max_length = 512 cố định: 35/118 Node hiện tại (30%) dài hơn 512 token, đúng các Node bảng và Điều dài, nên reranker không thấy dòng cần tìm. Đặt lại 2 048 token (dat_do_dai_toi_da() trong src/retriever.py) sửa được phần này: rerank tăng bộ 01 từ 25 lên 27 câu và dev đợt 2 từ 10 lên 14 câu. Lỗi còn lại là lỗi phiên bản: reranker chấm theo độ khớp nội dung nên Điều gốc của Quy chế 2021 đứng trên mục QĐ 3183 sửa

đổi chính Điều đó, và bộ 02 vẫn tụt từ 9 xuống 5 câu. Thêm bước lọc phiên bản trước và sau rerank đưa bộ 02 lên 7 câu, dev đợt 1 lên 20/20 và dev đợt 3 lên 14/15; R@1 gộp tăng từ 83,5% (chỉ rerank) lên 91,2%. Chi phí là khoảng 0,8 giây mỗi câu trên GPU 4 GB (trung vị), nhỏ so với thời gian sinh câu trả lời.

Lọc phiên bản cần dựa thêm vào điểm rerank. Lần đo đầu, rerank cùng lọc phiên bản làm dev đợt 2 tụt còn 12/15: TB 2567 chỉ bổ sung chứng chỉ PTE nhưng ghi là sửa đổi TB 2366/2451, nên bước đưa bản sửa đổi lên trước đẩy 2567 lên top-1 cho cả câu hỏi về IELTS, VSTEP, TOEFL; tương tự, QĐ 898 bị đẩy lên trên Quy định 3248 cho câu "khiếu nại điểm quá trình". Mục sửa đổi theo Điều (sua_doi_dieu) vẫn luôn được đưa lên vì biết chính xác phần bị sửa; quan hệ sửa đổi cả văn bản nay chỉ được đưa lên khi điểm rerank của văn bản mới đạt ít nhất 0,5 lần văn bản cũ. Trên dev, các lần đưa lên đúng (QĐ 898 Điều 5 về lệ phí và thời hạn phúc khảo) có tỉ lệ từ 0,92 trở lên, các lần sai đều dưới 0,35. Ngưỡng này được chọn trên tập dev nên phải kiểm lại trên tập kiểm chứng.

Hybrid BM25 không giúp. Giả thuyết là BM25 giúp các token hiếm như IELTS, TOEIC, VSTEP và số hiệu văn bản. Trên dev đợt 2 (15 câu quy đổi tiếng Anh), hybrid chỉ tăng từ 10 lên 11 câu đúng top-1, nhưng làm bộ 02 tụt từ 9 xuống 6 và dev đợt 3 từ 12 xuống 10; gộp 91 câu còn 76,9% so với 82,4% của vector. Khi đã có rerank, thêm hybrid không đổi kết quả câu nào mà chậm hơn. Trộn RRF với trọng số bằng nhau kéo lên các Node chứa nhiều tên chứng chỉ của văn bản khác khóa, còn reranker đọc cả câu nên đã phân biệt được tên chứng chỉ.

Kiểm định thống kê. Kiểm định McNemar chính xác trên top-1, gộp 91 câu: rerank so với rerank cùng lọc phiên bản là 0 câu so với 7 câu, $p = 0{,}016$; vector so với rerank cùng lọc là 3 so với 11, $p = 0{,}057$; vector so với rerank là 8 so với 9, $p = 1{,}000$; vector so với hybrid là 10 so với 5, $p = 0{,}302$. Chỉ tác dụng của bước lọc phiên bản đạt ngưỡng $p < 0{,}05$; mức tăng so với vector đơn thuần gần ngưỡng nhưng chưa đủ để khẳng định với 91 câu. Bộ 02 còn 3 câu trượt top-1, đều là lỗi truy hồi thật: hai câu hỏi khoảng điểm B+, D+ trên thang 10 lấy mục sửa đổi Điều 10 (bảng quy đổi sang thang 4) thay vì Điều 9, và câu hỏi khóa áp dụng cần Điều 2 của QĐ 3183.

8.4 Dùng ở đâu trong đồ án

Cả ba kỹ thuật đều có công tắc (use_rerank, loc_phien_ban, use_hybrid; biến môi trường RAG_RERANK, RAG_LOC_PHIEN_BAN; ô chọn trong giao diện Gradio).

Theo kết quả đo, profile server bật rerank và lọc phiên bản, tắt hybrid. make eval đo lại các cấu hình khi corpus thay đổi.

CHƯƠNG 9. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

9.1 Mô tả bài toán

Sinh viên SGU hỏi bằng tiếng Việt (thường viết tắt, sai chính tả, hỏi nối tiếp) về quy chế đào tạo, và nhận câu trả lời ngắn kèm văn bản và Điều nguồn để tự kiểm tra. Kho tài liệu gồm 20 văn bản đang áp dụng, trong đó có văn bản sửa đổi chỉ áp dụng cho một số khóa (QĐ 3183/2025 sửa Điều 9, 10 cho khóa từ 2025) và các bảng quy đổi chứng chỉ tiếng Anh khác nhau theo khóa tuyển sinh.

9.2 User story

Các câu chuyện người dùng dưới đây lấy từ câu hỏi thật trong nhật ký hỏi đáp:

- Là sinh viên, tôi muốn hỏi "thời gian tối đa hoàn thành khóa học" và nhận câu trả lời kèm Điều để tự kiểm tra.
- Là sinh viên, khi hỏi điều không có trong quy chế ("bao nhiêu tiền một tín chỉ"), tôi muốn hệ thống nói rõ là không tìm thấy.
- Là sinh viên khóa 2025, khi hỏi "8,4 hệ 10 thì hệ 4 được bao nhiêu", tôi muốn câu trả lời theo thang điểm áp dụng cho khóa của tôi.
- Là sinh viên, tôi muốn hỏi bằng câu không dấu hoặc sai chính tả ("xuất sắt", "dạt diều khiện") mà vẫn được trả lời.
- Là sinh viên, sau khi hỏi về cử nhân, tôi muốn hỏi tiếp "còn kỹ sư?" mà không phải lặp lại cả câu.

Story về khóa tuyển sinh được đáp ứng ở bước truy hồi khi câu hỏi nêu khóa (lọc phiên bản, Mục 8.2). Hai story cuối mới được đáp ứng một phần (BM25 bỏ dấu) hoặc chưa (không nhớ hội thoại).

9.3 Thiết kế

9.3.1 Sơ đồ pipeline

Pipeline thật gồm sáu giai đoạn (Hình 3.1, Chương 3): dữ liệu → Node → embedding/index → truy hồi → tổng hợp (ngữ cảnh + LLM + trích nguồn) → đánh giá.

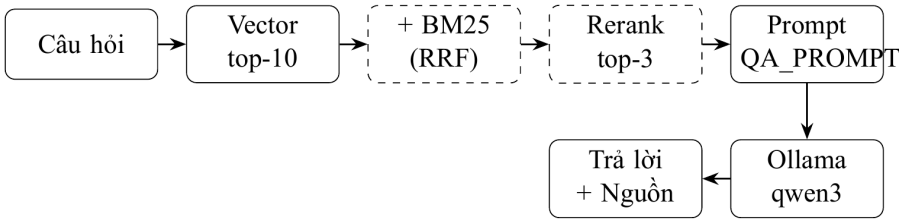
9.3.2 Mô hình dữ liệu

Bảng 9.1: Metadata của Node (chỉ khóa có giá trị mới được gắn).

Khóa metadata	**Ý nghĩa**	**Embed**	**Prompt**
nguon_trich	"Điều 9 — Quy chế …2021" (sinh tự động)	có	có
van_ban	Tên hiển thị của văn bản (src/corpus.py)	không	có
chuong	Tên Chương	không	có
dieu / sua_doi_dieu	Số Điều, hoặc số Điều được sửa đổi	không	có
phan	chinh / sua_doi / ban_hanh / noi_dung	không	không
phan_doan	Thứ tự đoạn khi một Điều bị cắt tiếp	không	có
file, file_goc, url	Truy vết về file tầng 2 và PDF gốc	không	không
so_hieu, ngay_ban_hanh, tinh_trang	Số hiệu, ngày ban hành, tình trạng hiệu lực (front matter)	không	có
sua_doi, bi_sua_doi_boi, thay_the, bi_thay_the_boi	Quan hệ giữa các văn bản, dùng cho lọc phiên bản	không	không
ap_dung_khoa_tu, ap_dung_khoa_den	Khóa tuyển sinh áp dụng; mục sửa đổi lấy theo ap_dung_khoa_tu_theo_dieu	không	không

Bộ câu hỏi truy hồi là JSON {"question", "dieu_dung"}; bộ chấm câu trả lời thêm pham_vi ("trong"/"ngoai"), dap_an và nguon.

9.3.3 Luồng truy vấn



Hình 9.1: Luồng truy vấn; khung nét đứt là bước tuỳ chọn; ở profile server, rerank và lọc phiên bản bật mặc định, hybrid và ngưỡng tương đồng tắt.

CHƯƠNG 10. TRIỂN KHAI, TÍCH HỢP VÀ MINH CHỨNG KẾT QUẢ

10.1 Tích hợp hệ thống

- **Dùng trực tiếp**: VectorStoreIndex, StorageContext, SentenceSplitter, QueryFusionRetriever, SentenceTransformerRerank, SimilarityPostprocessor, RetrieverQueryEngine, RetrieverEvaluator, các evaluator câu trả lời.
- **Viết lại / tự viết**: bước dữ liệu (OCR + Markdown hai tầng thay SimpleDirectoryReader), loader Node theo heading, BM25, client model server, bộ lọc phiên bản, độ đo thống kê.
- **Điểm vào**: src/pipeline.py (dòng lệnh), app/app.py (Gradio, ghi nhật ký câu hỏi), eval/run_batch_cau_hoi.py (chạy hàng loạt).
- **Thí nghiệm chỉ để ra quyết định**: so sánh vector/rerank/lọc phiên bản/hybrid (quyết định bật rerank cùng lọc phiên bản, bỏ hybrid), chọn các văn bản đang áp dụng thay vì nạp cả văn bản hết hiệu lực (văn bản cũ lấn át Quy chế 2021 khi truy hồi), chọn profile server làm cấu hình đo chính.

10.2 Kết quả đạt được

10.2.1 Truy hồi

Bảng 4.3 và 4.4 (Chương 4) cho chỉ số truy hồi; Bảng 4.2 cho top-*k* Node của các câu mẫu; Bảng 8.1 (Chương 8) so sánh các cấu hình nâng cao.

10.2.2 Câu trả lời

10.2.2.1 Cách đánh giá

Câu trả lời được chấm tay trên phiên hỏi đáp 56 câu (Mục 10.2.2.2). Để chấm tự động, nhóm viết eval/evaluate_answers.py để chấm tự động bằng chính các evaluator của LlamaIndex, chạy hàng loạt qua BatchEvalRunner:

- FaithfulnessEvaluator — câu trả lời có suy ra được từ ngữ cảnh truy hồi không;
- RelevancyEvaluator — câu trả lời có đúng câu hỏi không;
- CorrectnessEvaluator — so với đáp án chuẩn trong eval/cau_hoi_danh_gia.json;

- chế độ closed-book (LLM trả lời *không* có ngữ cảnh) làm mốc so sánh RAG với không RAG.

Bộ câu hỏi chấm câu trả lời gồm 13 câu, mỗi câu có trường pham_vi ("trong" / "ngoai") và đáp án chuẩn lấy từ bảng chấm tay ở Mục 10.2.2.2. Nhờ trường pham_vi, script tách **từ chối đúng** (câu ngoài phạm vi) khỏi **từ chối sai** (đáp án có trong ngữ cảnh nhưng hệ thống vẫn nói không tìm thấy) — chính là hai loại đã bị gộp thành một con số "52% từ chối" ở phiên chấm tay đầu tiên.

Trạng thái chạy. Script đã viết xong, kiểm thử import và logic nhận dạng từ chối, nhưng **chưa chạy** make eval-answers. Trong tuần 5 (05/10–09/10/2026), LLM đã chạy được qua Ollama trên Colab (scripts/setup_server_v2.sh), và nhóm dùng nó để chạy lại 56 câu hỏi của phiên trước với prompt mới (Mục 10.2.2.4) — vẫn chấm tay. Vì vậy Mục 10.2.2.2 trình bày kết quả chấm tay của phiên chạy trước (có ghi rõ nguồn), còn các bảng điểm faithfulness/relevancy/correctness sẽ được điền khi chạy make eval-answers. Script eval/tra_loi_dev.py ghi thêm số token vào/ra, độ trễ và số lượt gọi LLM lấy trực tiếp từ Ollama cho từng câu dev, để so cấu hình cũ (top-10) với cấu hình chốt (rerank top-3 cùng lọc phiên bản).

10.2.2.2 Kết quả chấm tay (phiên chạy 56 câu, profile server có rerank)

Nhật ký eval/results/test-basic.md ghi một phiên chạy src/pipeline.py: 56 câu hỏi do nhóm tự đặt theo cách sinh viên thật sự hỏi, gồm cả viết tắt ("bao nhiêu chỉ"), sai chính tả ("xuất sắt"), câu hỏi nối tiếp ("còn kỹ sư") và câu ngoài phạm vi. Hệ thống trả lời "không tìm thấy" ở 29/56 câu (52%) và đưa ra câu trả lời ở 27 câu.

Bảng 10.1: Một số câu trả lời trong nhật ký phiên chạy đầu tiên, đối chiếu với văn bản Quy chế 2021. Đây là số liệu chấm tay, không sinh lại được bằng lệnh. *Phiên chấm đầu tiên ghi câu này là "Sai một phần"; khi đối chiếu lại khoản 4 Điều 9 trong tuần 5 (05/10–09/10/2026) thì câu trả lời của hệ thống là đúng, còn đáp án tham chiếu đã nhầm hai trường hợp học lại và học cải thiện.

Câu hỏi	**Hệ thống trả lời**	**Quy chế 2021 quy định**	**Đánh giá**
Học cử nhân cần bao nhiêu tín chỉ	Tối thiểu 120 tín chỉ	120 tín chỉ (Điều 2)	Đúng
Muốn được B cần bao nhiêu điểm	Từ 7,0 đến 8,4	B: 7,0–8,4 (Điều 9)	Đúng

Câu hỏi	**Hệ thống trả lời**	**Quy chế 2021 quy định**	**Đánh giá**
Cảnh báo kỳ 1 và 3, kỳ 2 bình thường có bị buộc thôi học	Không, vì hai lần không liên tiếp	Buộc thôi học khi quá hai lần cảnh báo *liên tiếp* (Điều 11)	Đúng
Mua bằng tiếng Anh giả nộp cho trường	Không được; buộc thôi học, thu hồi bằng	Như câu trả lời (Điều 19)	Đúng
8,4 hệ 10 thì hệ 4 được bao nhiêu	3,4 điểm	Khóa trước 2025: 8,4 là B, quy đổi 3,0; khóa 2025 trở đi (QĐ 3183): 8,4 là B+, quy đổi 3,5 (Điều 9, 10)	Sai
Khối lượng học kỳ chính	Tối thiểu 30 tuần	2 học kỳ chính *cộng lại* tối thiểu 30 tuần (Điều 6)	Sai
Học bị F thì làm gì	Học lại; điểm lần học cuối là điểm chính thức	Học phần không đạt phải học lại, điểm *lần học cuối* là điểm chính thức; "cao nhất" chỉ áp dụng khi học cải thiện (Điều 9 khoản 4)	Đúng*
Học điểm P thì sao	P không tính vào điểm TB, "quy đổi thành 0"	P không được tính vào điểm TB (Điều 10 khoản 3), không quy đổi	Sai một phần
Ra trường xuất sắc cần bao nhiêu điểm	Không tìm thấy	Từ 3,6 đến 4,0 (Điều 10), và Điều 10 *có* trong 3 Điều nguồn	Từ chối sai
Còn kỹ sư (hỏi tiếp câu cử nhân)	Không tìm thấy	150 tín chỉ; hỏi lại đầy đủ thì trả lời đúng	Từ chối sai
Điểm rèn luyện là gì	Không tìm thấy	Không thuộc Quy chế đào tạo (văn bản riêng của CTSV)	Từ chối đúng
Bao nhiêu tiền một tín chỉ	Không tìm thấy	Không thuộc Quy chế đào tạo	Từ chối đúng

10.2.2.3 Phân tích

Prompt chống bịa hoạt động, nhưng hơi quá tay. Với câu hỏi ngoài phạm vi, hệ thống từ chối thay vì bịa — đúng mục tiêu. Nhưng tỷ lệ từ chối 52% cao hơn tỷ lệ câu thật sự ngoài phạm vi: có câu mà thông tin *đã nằm trong ngữ cảnh* vẫn bị từ chối, như câu hỏi về điểm ra trường loại xuất sắc (viết sai chính tả "xuất sắt"). Đây chính là lý do phải tách "từ chối đúng" khỏi "từ chối sai" như thiết kế ở Mục 10.2.2.

Lỗi sai tập trung ở phép tính và chi tiết số. Các câu trả lời sai đều có Điều đúng trong ngữ cảnh (truy hồi đúng), nhưng LLM đọc sai hoặc suy diễn thêm: tự "quy đổi" 8,4

thành 3,4 thay vì tra bảng, hay đọc "30 tuần cho 2 học kỳ" thành "30 tuần mỗi học kỳ". Đây là loại lỗi nguy hiểm nhất vì câu trả lời vẫn trích dẫn đúng Điều. QA_PROMPT có thêm hai ràng buộc nhắm đúng loại lỗi này: *trích nguyên văn câu quy định trước khi kết luận* và *không tự quy đổi thang điểm, cần tra bảng thì chép đúng hàng của bảng*. Hiệu quả của hai ràng buộc này cần được đo bằng make eval-answers trên máy có LLM, và được ghi nhận là việc còn lại.

Câu trả lời không ổn định — đã xử lý ở mức mã nguồn. Nhật ký phiên đầu ghi nhận cùng một câu hỏi chỉ khác một dấu hỏi chấm mà lần đầu từ chối, lần sau trả lời "Không". Nguyên nhân là Ollama(...) không đặt temperature, nên mô hình lấy mẫu ngẫu nhiên. Hiện make_llm() đặt temperature=0 và seed=42; đây là sửa đổi đã kiểm tra ở mức mã nguồn nhưng cần một phiên chạy LLM mới để xác nhận bằng số.

Không có ngữ cảnh hội thoại. RetrieverQueryEngine gửi từng câu hỏi độc lập, nên câu nối tiếp "còn kỹ sư" bị từ chối. Sinh viên thường hỏi nối tiếp, nên đây vẫn là hạn chế lớn nhất ở tầng sinh.

10.2.2.4 Chạy lại 56 câu với prompt mới (ngày 08/10/2026)

Cách chạy. Cùng 56 câu hỏi của phiên trước (eval/results/cau-hoi01.md) được chạy lại bằng eval/run_batch_cau_hoi.py (bản notebook: notebooks/chay_cau_hoi01.ipynb), profile server, qwen3:8b trên Colab, temperature=0, seed=42. Kết quả ghi ở eval/results/cau-hoi01-result.md (phiên bản trong commit be7b81e, ngày 08/10). Prompt dùng trong lần chạy này đã có hai ràng buộc "trích nguyên văn" và "không tự quy đổi" nhưng *chưa* có chú giải thang điểm ở Mục 4.5.2.

Lưu ý khi so sánh. Hai phiên khác nhau ở *hai* điểm cùng lúc: prompt mới, và cấu hình ngữ cảnh — phiên trước bật rerank (LLM chỉ nhận 3 Node), phiên này tắt rerank (LLM nhận 10 Node). Vì vậy chênh lệch dưới đây không quy được riêng cho prompt; muốn tách phải chạy thêm một phiên chỉ đổi một yếu tố.

Kết quả tổng. Số câu trả lời "không tìm thấy" giảm từ 29/56 (52%) xuống 23/56 (41%).

Bảng 10.2: Một số câu của phiên chạy lại ngày 08/10, chấm tay theo văn bản trong data/processed.

Câu hỏi	Trả lời (phiên 08/10)	Quy chế quy định	So với phiên trước
Ra trường xuất sắc cần bao nhiêu điểm	ĐTB tích lũy 3,6–4,0	3,6–4,0 (Điều 10 khoản 4)	Từ chối sai → **Đúng**
8,4 hệ 10 thì hệ 4 được bao nhiêu	3,5	3,5 với khóa 2025 trở đi (B+); 3,0 với khóa trước	Sai → Đúng một phần (không nêu khóa)
Điểm D+ ứng với khoảng nào trên thang 10	4,8–5,4	4,8–5,4 (QĐ 3183, Điều 1 khoản 3)	(câu mới) Đúng
Giỏi tiếng Anh nhưng không có chứng chỉ có ra trường được không	Được xét nếu đủ điều kiện; chưa đạt chuẩn ngoại ngữ thì có 3 năm để hoàn thiện	Như câu trả lời (Điều 13 khoản 4)	Đúng
Không có chứng chỉ tiếng Anh có ra trường được không	Không, dẫn Điều 3 về học chương trình thứ hai ngành Ngôn ngữ Anh	Điều 13 khoản 4, như câu trên	**Sai** (lấy nhầm văn bản QĐ 810)
Học bị F thì làm gì	F quy đổi 0 và "không được tính vào điểm trung bình"	F quy đổi 0 và *được* tính (Điều 10 khoản 2–3)	Đúng → **Sai**
Khối lượng học kỳ chính	Mỗi học kỳ chính tối thiểu 30 tuần	2 học kỳ chính cộng lại tối thiểu 30 tuần (Điều 6)	Sai (không đổi)
Rớt một môn, các kỳ còn lại 4.0 có được xuất sắc không	Không tìm thấy	Chỉ bị giảm hạng nếu học lại vượt 5% tổng tín chỉ (Điều 13 khoản 3)	Từ chối sai
Còn kỹ sư (hỏi tiếp)	Không tìm thấy	150 tín chỉ	Từ chối sai (không đổi)
Mua bằng tiếng Anh giả (hai cách hỏi)	Một cách hỏi: đúng, dẫn Điều 19 khoản 3; cách kia: không tìm thấy	Buộc thôi học, thu hồi bằng (Điều 19 khoản 3)	Không ổn định theo cách diễn đạt
Bao nhiêu tiền 01 tín	In ra nguyên chuỗi /no_think	Ngoài phạm vi	Từ chối đúng → **Lỗi**

Phân tích.

- **Ít từ chối sai hơn**, rõ nhất ở câu "xuất sắc" — trước đây bị từ chối dù Điều 10 có trong ngữ cảnh. Do ngữ cảnh tăng từ 3 lên 10 Node cùng lúc với đổi prompt, chưa kết luận được yếu tố nào có tác dụng.
- **Phạm vi hiệu lực theo khóa tuyển sinh là nguồn sai mới.** QĐ 3183 chỉ áp dụng thang điểm có B+, C+, D+ cho khóa tuyển sinh từ 2025 (Điều 2 của QĐ). Hệ thống

trả lời theo một trong hai bảng mà không nói bảng đó áp dụng cho khóa nào: "8,4 → 3,5" đúng với khóa 2025, còn "B là 7,0–8,4" chỉ đúng với khóa trước 2025. Prompt hiện chỉ nói "có văn bản sửa đổi thì dùng nội dung đã sửa đổi", tức là ngầm giả định mọi sinh viên đều thuộc khóa mới.

- **Ngữ cảnh rộng hơn cũng kéo theo văn bản nhiễu.** Câu "không có chứng chỉ tiếng Anh" lấy Điều 3 của QĐ 810 (điều kiện học chương trình thứ hai ngành Ngôn ngữ Anh) làm căn cứ, trong khi Điều 13, căn cứ đúng, cũng có trong ngữ cảnh. Câu về điểm F sai ở chi tiết "không tính vào điểm trung bình", nhầm F với các điểm chữ không quy đổi ở khoản 3 Điều 10.
- **Không ổn định theo cách diễn đạt.** temperature=0 làm *cùng một* câu hỏi cho cùng câu trả lời, nhưng hai cách hỏi gần nhau vẫn có thể cho một câu đúng và một câu "không tìm thấy". Tính tất định không thay được độ vững theo diễn đạt.
- **Rò chỉ thị /no_think.** Ở một câu ngoài phạm vi, Qwen3 trả về đúng chuỗi điều khiển /no_think thay cho câu trả lời. tests/test_llm.py nay kiểm tra câu trả lời không chứa /no_think, và eval/compare_results.py đếm chuỗi này như một câu từ chối; bộ lọc chuỗi ở đầu ra chưa làm.

10.2.2.5 Sinh câu trả lời trên cấu hình chốt (ngày 17/10/2026)

Cách chạy. eval/tra_loi_dev.py sinh câu trả lời cho 50 câu dev đợt 1–3 bằng cấu hình chốt (vector top-10, lọc phiên bản, rerank đọc 2 048 token, giữ 3 Node), LLM qwen3:8b qua Ollama trên Colab (GPU Tesla T4), temperature = 0, seed = 42; embedding và reranker chạy trên máy (GPU 4 GB). Số token và thời gian sinh lấy trực tiếp từ phản hồi của Ollama (prompt_eval_count, eval_count, eval_duration), không ước lượng. Hai câu của đợt 2 không có câu trả lời vì tunnel Cloudflare trả lỗi 5xx cả bốn lần thử; hai câu này không tính vào bảng. Đợt 2 và đợt 3 có đáp án trong file câu hỏi nên được chấm tay theo đáp án; đợt 1 chưa có đáp án.

Bảng 10.5: Sinh câu trả lời trên cấu hình chốt, 50 câu dev (eval/results/tra-loi-dev-dot*-moi-2026-10-17.json).

Bộ	Có trả lời	Thời gian trung vị / p90 (s)	Token vào TB	Token ra TB	Token/s	Lượt gọi LLM tối đa	Chấm theo đáp án
Dev đợt 1	20/20	8,8 / 14,8	2 738	186	24,8	1	Chưa có

							đáp án
Dev đợt 2	13/15	8,9 / 14,4	3 051	142	27,3	1	11 đúng, 2 sai
Dev đợt 3	15/15	8,1 / 16,0	2 469	150	24,7	1	13 đúng, 2 đúng một phần

- **Một lượt gọi LLM cho mọi câu.** Với 3 Node sau rerank, prompt khoảng 2 500–3 000 token, dưới hạn mức của chế độ compact, nên không câu nào phải gọi thêm lượt refine bằng prompt tiếng Anh mặc định, đúng như tính toán ở Mục 4.2.2. Độ trễ khoảng 9 giây mỗi câu gần như toàn bộ là thời gian sinh trên T4 (khoảng 25 token/giây); truy hồi và rerank dưới 1 giây.
- **Kết quả chấm.** Trên 28 câu có đáp án và có câu trả lời: 24 đúng, 2 đúng một phần, 2 sai. Không câu nào rò chuỗi /no_think; chỉ một câu trả lời có cụm "không tìm thấy".
- **Hai câu sai đều là lỗi tra bảng.** Câu "khóa 2021, PTE Academic 50 điểm" được trả lời theo bảng quy đổi sang thang 4 của Quy chế 2021 (3 điểm) thay vì 8 điểm của TB 2567, dù TB 2567 đứng đầu ngữ cảnh. Câu "khóa 2024, Linguaskill 155 điểm" được trả lời là không được quy đổi vì mô hình chỉ đọc dòng "≥ 159", trong khi 155 thuộc mức 150–158 (9 điểm). Đây là loại lỗi mà công cụ tra bảng trên CSV quy đổi (Mục 12.1) nhắm tới.
- **Hai câu đúng một phần.** Câu chuẩn đầu ra ngoại ngữ 2 chỉ nêu Bậc 3/6, thiếu danh sách chứng chỉ theo từng ngôn ngữ. Câu so sánh thời gian tối đa với QĐ 736/2014 trả lời đúng phần Quy chế 2021 và nói không tìm thấy phần QĐ 736; điều này đúng với thiết kế vì QĐ 736 xung đột với Quy chế 2021 nên không được nạp (Mục 4.1.5).
- **Chưa so được với cấu hình cũ.** Lần chạy cấu hình cũ (top-10, không rerank, khoảng 7 000 token vào) dừng ngay ở câu khởi động vì Ollama trên Colab báo "model runner has unexpectedly stopped" (lỗi 500). Phép so sánh cũ và mới, cùng hai câu bị lỗi kết nối, sẽ chạy lại khi có server ổn định.

10.2.3 So sánh trước – sau các tối ưu

Bảng 10.3: Các thay đổi đã đo hoặc kiểm chứng được.

Vấn đề	Trước	Sau
Điều vắt qua hai trang	Bị chia đôi (mỗi trang một Document)	Một Node mỗi Điều (Markdown hai tầng)
Điều 1–3 của quyết định ban hành	Trùng nhãn với Điều 1–3 của quy chế	Tách phan="ban_hanh"
Metadata rỗng trong embedding	Có (chuong:, dieu: rỗng)	Chỉ nguon_trich
Dùng nhầm chỉ mục cũ	Có thể, nếu quên --rebuild	Fingerprint tự build lại
Câu trả lời giữa các lần hỏi	Không ổn định	temperature=0, seed=42
Embedding + reranker trên GPU 4 GB	CUDA out of memory	fp16, khoảng 2,2 GB
Tỷ lệ "không tìm thấy" (56 câu)	52%	41% (đổi prompt và top-k cùng lúc)
Reranker chỉ đọc 512 token đầu mỗi cặp	35/118 Node bị cắt mất phần cần tìm	Đọc 2 048 token, không Node nào bị cắt
Rerank đẩy Điều gốc lên trên mục sửa đổi (bộ 02, top-1)	5/10 câu đúng (chỉ rerank)	7/10 câu đúng (rerank cùng lọc phiên bản)
R@1 trên 91 câu dev	82,4% (vector)	91,2% (cấu hình chốt, Chương 8)

10.2.4 Hướng dẫn chạy lại

Đoạn mã 10.1: Các lệnh chạy lại (từ thư mục gốc repo).

```
python3 -m venv venv && venv/bin/pip install -r requirements.txt
bash scripts/pull_models.sh server          # hoặc bash scripts/setup_server_v2.sh trên Colab
RAG_PROFILE=server venv/bin/python src/index_builder.py
RAG_PROFILE=server venv/bin/python src/pipeline.py      # hỏi đáp dòng lệnh
RAG_PROFILE=server venv/bin/python app/app.py           # giao diện Gradio
make test && make eval && make baseline                 # kiểm thử và số liệu truy hồi
```

10.3 Bảng phủ kỹ thuật

Bảng phản ánh trạng thái tuần 6 (16/10/2026); bản cuối sẽ khóa cùng commit niêm phong, nộp ít nhất 24 giờ trước buổi bảo vệ. Chỉ khai những gì có trong commit đó.

Bảng 10.4: Bảng phủ kỹ thuật.

TT	Nội dung	Tầng	Chức năng	File	Minh chứng	Trạng thái
1	OCR + Markdown hai tầng	1A	Nạp quy chế	scripts/extract_to_md.py, clean_md.py	20 văn bản	Dùng
2	Node theo Điều	1A +1 B	Đơn vị truy hồi	src/processed_loader.py	118 Node, kiểm thử	Dùng
3	Embedding + index + persist	1A +1 B	Lập chỉ mục	src/index_builder.py	4 s / 0,5 s (GPU)	Dùng
4	Retriever top-k	1A	Truy hồi	src/retriever.py	Bảng 4.2	Dùng
5	Query engine + trích nguồn	1A +1 B	Hỏi đáp	src/query_engine.py	Dòng "Nguồn"	Dùng
6	Recall@k, MRR, nDCG, CI	1A	Kiểm chứng	eval/evaluate.py, metrics.py	R@1 91,2% trên 91 câu dev	Dùng
7	Từ chối ngoài corpus	1B	Hỏi đáp	src/query_engine.py	41% "không tìm thấy"	Chưa đo tỷ lệ đúng
8	Hybrid BM25	3	Truy hồi	src/bm25.py	Bảng 8.1	Có, tắt
9	Rerank cross-encoder	3	Truy hồi	src/retriever.py	Bảng 8.1	Dùng (mặc định bật)
10	Model server HTTP	3	Hạ tầng	src/model_server.py	Chạy trên Colab	Dùng
11	.env, 2 profile	2	Cấu hình	src/config.py, .env.example	—	Dùng
12	Unit test + CI	2	Kiểm thử	tests/, .github/workflows/ci.yml	54 kiểm thử	Dùng
13	Giao diện Gradio	1B	Demo	app/app.py	Nhật ký câu hỏi	Chạy thử
14	Lọc phiên bản theo metadata hiệu lực	3	Truy hồi	src/phien_ban.py	Bảng 8.1, tests/test_phien_ban.py	Dùng

CHƯƠNG 11. ĐÁNH GIÁ VÀ NHÌN LẠI

11.1 So sánh với mục tiêu ban đầu

Bảng 11.1: Đối chiếu mục tiêu và kết quả.

Mục tiêu (Lời mở đầu)	Kết quả	Minh chứng
6 mục lõi Tầng 1A	Đạt 6/6 (mục 6: đánh giá truy hồi đạt; đánh giá câu trả lời bằng evaluator chưa chạy)	Bảng 4.1
Custom node parser, persistent index, trích nguồn	Đạt	Chương 5–7
Từ chối câu ngoài corpus	Một phần	Chưa đo tỷ lệ đúng
Thí nghiệm chunking	Chưa	Mục 4.2
Hybrid, rerank (Tầng 3)	Đạt, có số liệu	Chương 8
Kiểm thử + CI (Tầng 2)	Đạt	54 kiểm thử
Chấm câu trả lời tự động	Chưa chạy	Mục 10.2.2

Các quyết định đổi hướng.

- **Từ OCR theo trang sang Markdown hai tầng**: vì Điều bị chia đôi giữa hai trang và Điều của quyết định ban hành trùng nhãn (Chương 5).
- **Từ một văn bản lên 20 văn bản đang áp dụng**: tuần 4 có 6 văn bản để trả lời được nội dung sửa đổi, tuần 6 nạp thêm quy đổi tiếng Anh, thi và phúc khảo, học vụ; văn bản hết hiệu lực không nạp vì lấn át văn bản đang áp dụng khi truy hồi.
- **Chọn profile server làm cấu hình chính, LLM chạy từ xa**: GPU 4 GB không chạy nổi qwen3:8b cùng embedding; LLM chạy trên máy GPU thuê hoặc Colab (scripts/setup_server_v2.sh), embedding chạy local ở fp16 hoặc CPU.
- **Bật lại rerank, bỏ hybrid**: trên 54 Node, rerank không giúp vì reranker chỉ đọc 512 token đầu (28% Node bị cắt mất phần cần tìm) và đẩy Điều gốc lên trên mục sửa đổi. Sau khi cho reranker đọc 2 048 token và thêm bước lọc phiên bản, R@1 trên 91 câu dev tăng từ 82,4% lên 91,2%; hybrid vẫn không giúp (Chương 8).

11.2 Hạn chế

- **Độ mạnh thống kê của bộ câu hỏi.** Tập dev nay có 91 câu và tập kiểm chứng 60 câu do hai người gán nhãn độc lập (κ = 1,00), nhưng khoảng tin cậy vẫn rộng (bộ 02 chỉ có 10 câu) và phần lớn so sánh cặp chưa đạt $p < 0,05$ (chỉ rerank so với rerank cùng lọc phiên bản đạt p = 0,016). Cần mở rộng lên khoảng 150–300 câu, phân tầng theo loại câu hỏi.
- **Corpus nhỏ.** 20 văn bản / 118 Node. Kết luận về rerank, lọc phiên bản và hybrid ở Chương 8 mới đo trên tập dev; cần đo lại trên tập kiểm chứng và khi kho lớn hơn.
- **Chất lượng OCR.** Văn bản quan trọng nhất (QĐ 3183/2025) có chất lượng OCR thấp nhất (94,9% âm tiết hợp lệ) và vẫn còn lỗi như "Sửa đồi".
- **Metadata quan hệ chưa phân biệt sửa đổi với bổ sung.** Số hiệu, ngày ban hành, tình trạng, quan hệ sửa đổi và khóa áp dụng đã được điền và xác minh cho mọi văn bản, và bước lọc phiên bản dùng chúng. Nhưng TB 2567 (chỉ bổ sung chứng chỉ PTE) cũng ghi là sửa đổi TB 2366/2451, nên bộ lọc phải dựa thêm vào điểm rerank để không đẩy nhầm văn bản bổ sung lên trước (Mục 8.2).
- **Đánh giá câu trả lời chưa chạy tự động.** Script đã có và LLM đã chạy được trên Colab, nhưng make eval-answers chưa chạy; câu trả lời mới chỉ được chấm tay.
- **Hạ tầng đo.** Số liệu truy hồi đo ngày 17/10 trên GPU 4 GB; baseline và một số bảng ở Chương 4 đo ngày 08–09/10 trên CPU với corpus 54 Node, nên thời gian tuyệt đối giữa hai nhóm số không so sánh được.
- **Tầng sinh.** Không nhớ hội thoại, không có công cụ tra bảng/quy đổi điểm, chỉ một lượt gọi LLM trên ngữ cảnh đã truy hồi.
- **Khóa tuyển sinh mới được xử lý ở bước truy hồi.** Khi câu hỏi nêu khóa, bước lọc phiên bản bỏ văn bản không áp dụng cho khóa đó. Khi câu hỏi không nêu khóa, LLM vẫn có thể trả lời theo một bảng mà không nói bảng đó áp dụng cho khóa nào (Mục 10.2.2.4).
- **Chú giải thang điểm trong prompt chưa được đo.** Đây là kiến thức riêng của quy chế SGU đặt trong prompt (Mục 4.5.2), và chưa có phiên chạy nào xác nhận nó sửa được lỗi.

11.3 Phân tích lỗi

11.3.1 Lỗi 1: LLM nhầm thang điểm hệ 10 với hệ 4

- **Hiện tượng**: "A thì được bao nhiêu điểm hệ 10" → "A được quy đổi thành 4,0 điểm hệ 10" (nhật ký Gradio, ngày 08/10, hai lần hỏi).
- **Phát hiện**: đọc nhật ký questions_log.jsonl; top-k (Bảng 4.2) cho thấy truy hồi đúng, lỗi nằm ở tầng sinh.
- **Nguyên nhân và cách sửa**: Mục 4.5.2. Chưa đo lại sau khi sửa.

11.3.2 Lỗi 2: CUDA out of memory khi nạp embedding cùng reranker

- **Hiện tượng**: trên card 4 GB, profile server chiếm khoảng 3,5 GB sau khi nạp embedding và báo hết bộ nhớ khi nạp reranker.
- **Nguyên nhân**: SentenceTransformerRerank không nhận dtype, luôn nạp fp32.
- **Cách sửa**: nạp reranker trên CPU, ép fp16 rồi mới chuyển lên GPU:

Đoạn mã 11.1: Nạp reranker fp16 (make_local_reranker() trong src/retriever.py).

```
reranker = SentenceTransformerRerank(model=cfg.reranker_model_name, top_n=cfg.reranker_top_n,
                                     device="cpu" if fp16 else device)
if fp16:
    reranker._model.half().to(device)
    reranker.device = device
```

Sau khi sửa, cả hai mô hình cùng nằm trên GPU, tổng khoảng 2,2 GB (Mục 3.1.4).

11.4 AI Disclosure

(Chưa làm — phần này viết ở bản báo cáo cuối, dựa trên nhật ký dùng AI của nhóm). Nội dung sẽ gồm:

1. Công cụ AI đã dùng và dùng vào việc gì.
2. Phần có tỷ trọng AI sinh cao (ghi file / chức năng).
3. Ít nhất hai tình huống AI trả kết quả *sai*: prompt, AI trả về gì, sai ở đâu, phát hiện thế nào, sửa ra sao.
4. Tự đánh giá: nếu bỏ AI, phần nào nhóm vẫn tự viết lại được.

CHƯƠNG 12. HƯỚNG PHÁT TRIỂN

12.1 Cải tiến sản phẩm

Còn lại, theo thứ tự ưu tiên:

1. **Chạy** make eval-answers (LLM trên Colab) để có số faithfulness/relevancy/correctness, và so chế độ RAG với closed-book. Đây là việc còn thiếu duy nhất của phần đánh giá. Cùng đợt chạy đó, chạy lại 56 câu theo từng yếu tố một (chỉ đổi prompt, chỉ đổi top-*k* đưa vào LLM, bật/tắt RAG_THINKING) để tách tác dụng của từng thay đổi ở Mục 10.2.2.4.
2. **Xử lý khóa tuyển sinh**: đã gắn metadata khóa áp dụng (ap_dung_khoa_tu_theo_dieu) và lọc theo khóa ở bước truy hồi. Còn lại: yêu cầu câu trả lời về điểm nêu rõ áp dụng cho khóa nào (hoặc nêu cả hai bảng) khi câu hỏi không nêu khóa, và gắn câu dẫn "thang điểm 10"/"quy đổi để tính điểm trung bình" vào mục sửa đổi để thay chú giải đang đặt cứng trong prompt.
3. **Mở rộng bộ câu hỏi** lên vài trăm câu, có nhãn loại câu (trong/ngoài phạm vi, viết tắt, nối tiếp, cần tra bảng); tập kiểm chứng 60 câu với hai người gán nhãn độc lập là bước đầu.
4. **Phân biệt sửa đổi với bổ sung** trong metadata quan hệ (ví dụ thêm khóa bo_sung), để bước lọc phiên bản không phải dựa vào điểm rerank khi quyết định đưa văn bản nào lên trước.
5. **So câu trả lời của cấu hình cũ với cấu hình chốt** bằng eval/tra_loi_dev.py: so cấu hình cũ (top-10, không rerank) với cấu hình chốt, lấy token, độ trễ và số lượt gọi LLM từ Ollama; nếu còn câu phải gọi LLM nhiều lượt thì đặt refine_template tiếng Việt cùng các ràng buộc của QA_PROMPT.
6. **Small-to-big**: cắt thêm Node con theo khoản, truy hồi trên Node con rồi đưa Điều cha cho LLM (HierarchicalNodeParser + AutoMergingRetriever) — nhắm đúng lỗi "đọc sai chi tiết số" trong Điều dài.
7. **Chat engine có ngữ cảnh** (CondensePlusContextChatEngine) cho câu hỏi nối tiếp; giao diện Gradio đã có sẵn lịch sử chat.

8. **Công cụ tra bảng quy đổi điểm** (FunctionTool + agent) thay vì để LLM tự tính. Dữ liệu cho công cụ này đã có: 5 CSV quy đổi chứng chỉ tiếng Anh đã đối chiếu với PDF gốc (data/bang_quy_doi/).
9. **Vector store ngoài** (Chroma/Qdrant) khi corpus vượt vài trăm Node, để lọc metadata phức tạp và cập nhật từng phần.

12.2 Định hướng bản thân

(Chưa làm — từng thành viên viết ở bản báo cáo cuối).

12.3 Kết luận

Trên corpus 20 văn bản / 118 Node, cấu hình chốt (vector top-10, lọc phiên bản theo metadata hiệu lực, rerank đọc 2 048 token, giữ 3 Node) đạt Recall@1 = 91,2% trên 91 câu dev, so với 82,4% của truy hồi vector đơn thuần; so với rerank không lọc, mức tăng có ý nghĩa thống kê ($p = 0{,}016$). Hai phát hiện đến từ việc đo: reranker từng bị đánh giá là có hại thực ra chỉ đọc 512 token đầu mỗi Node, và rerank chỉ có ích khi đi cùng bước lọc phiên bản; hybrid BM25 không giúp ở quy mô này. Chỉ số tự tính khớp với RetrieverEvaluator của LlamaIndex, và bản dựng lại phần truy hồi bằng LangChain cho kết quả trùng khớp trên cả 91 câu. Điểm yếu còn lại nằm ở tầng sinh (LLM đọc sai số liệu trong Điều dài, chưa nhớ hội thoại, trên 28 câu dev có đáp án, cấu hình chốt trả lời đúng 24 câu và sai 2 câu, cả hai đều do tra bảng quy đổi), ở bộ 02 (7/10 câu đúng ở top-1) và ở độ mạnh thống kê của bộ câu hỏi.

TÀI LIỆU THAM KHẢO

[1] LlamaIndex Team, "LlamaIndex Documentation (phiên bản 0.14.24)". https://docs.llamaindex.ai/, 2025.

[2] LlamaIndex Team, "run-llama/llama_index: Mã nguồn thư viện". https://github.com/run-llama/llama_index, 2025.

[3] P. Lewis *và c.s.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks", trong *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[4] Y. Gao *và c.s.*, "Retrieval-Augmented Generation for Large Language Models: A Survey", *arXiv preprint arXiv:2312.10997*, 2023.

[5] N. Reimers và I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks", trong *Proceedings of EMNLP-IJCNLP*, 2019.

[6] N. Muennighoff, N. Tazi, L. Magne, và N. Reimers, "MTEB: Massive Text Embedding Benchmark", *arXiv preprint arXiv:2210.07316*, 2022.

[7] S. Robertson và H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond", *Foundations and Trends in Information Retrieval*, vol 3, số p.h 4, tr 333–389, 2009.

[8] N. F. Liu *và c.s.*, "Lost in the Middle: How Language Models Use Long Contexts", *Transactions of the Association for Computational Linguistics (TACL)*, 2024.

[9] V. Karpukhin *và c.s.*, "Dense Passage Retrieval for Open-Domain Question Answering", trong *Proceedings of EMNLP*, 2020.

[10] Ollama, "Ollama: chạy mô hình ngôn ngữ lớn cục bộ". https://ollama.com/, 2025.

[11] Qwen Team, "Qwen3 Technical Report". https://qwenlm.github.io/blog/qwen3/, 2025.

[12] Tesseract OCR, "Tesseract Open Source OCR Engine". https://github.com/tesseract-ocr/tesseract, 2024.

[13] BKAI Foundation Models, "bkai-foundation-models/vietnamese-bi-encoder (model card)". https://huggingface.co/bkai-foundation-models/vietnamese-bi-encoder, 2023.

[14] AITeamVN, "AITeamVN/Vietnamese_Embedding (model card)". https://huggingface.co/AITeamVN/Vietnamese_Embedding, 2024.

[15] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, và I. Gurevych, "BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models", trong *NeurIPS Datasets and Benchmarks Track*, 2021.

[16] R. Nogueira và K. Cho, "Passage Re-ranking with BERT", *arXiv preprint arXiv:1901.04085*, 2019.

PHỤ LỤC A. NHẬT KÝ CÔNG VIỆC HÀNG TUẦN

Mỗi dòng phải khớp với commit trong repo. Cột "Người làm" lấy theo tác giả commit.

Bảng A.1: Nhật ký công việc theo tuần, lập từ git log (tuần 5: commit be7b81e ngày 08/10 và các thay đổi chưa commit). Tên tài khoản git solozy do nhóm đối chiếu với thành viên.

Tuần	Công việc đã làm	Người làm	Khó khăn / kết quả
1–2	Lập nhóm, chọn công nghệ; cài môi trường, Hello World	—	Chưa có commit trong repo
3 (21–27/09)	Khởi tạo repo, dữ liệu Quy chế 2021; node_parser.py tách theo Điều; sửa check_model_memory.py; bộ câu hỏi 01 và 02; dữ liệu sổ tay sinh viên; nháp báo cáo (PR #1–#6)	Phan Hữu Kha, Võ Thị Như Ý, Trần Hà Thanh Thùy, Huỳnh Thanh Tiến,	Pipeline một văn bản chạy được
4 (28/09–04/10)	Cấu hình 2 profile, RAG cơ bản, tiền xử lý Markdown hai tầng, processed_loader.py, fingerprint chỉ mục, model server; báo cáo tuần 4	Huỳnh Thanh Tiến	6 văn bản, 54 Node
5 (05–09/10)	Chạy LLM + model server trên Colab; hybrid BM25, baseline, kiểm định thống kê; 26 kiểm thử + CI; Gradio có nhật ký câu hỏi; chạy lại 56 câu; chú giải thang điểm trong prompt, RAG_THINKING; cập nhật Chương 3–5	Huỳnh Thanh Tiến, Phan Hữu Kha	Truy hồi R@1 81% / 90%; rerank, hybrid không giúp; "không tìm thấy" 52% → 41%; phát hiện lỗi nhầm hệ 10/hệ 4 và lỗi không phân biệt khóa tuyển sinh
6 (12–16/10)	Nạp đợt 2–3, metadata hiệu lực, CSV bảng quy đổi và đối chiếu, câu dev đợt 3 (D1–D5); tập kiểm chứng 60 câu, gán nhãn độc lập, độ đồng thuận (K1–K3); reranker 2 048 token, lọc phiên bản, luật chấm, đo chốt cấu hình, so sánh LangChain (C1–C5, R10); sửa báo cáo theo nhận xét (PR #16–#21)	Phan Hữu Kha, Võ Thị Như Ý, Trần Hà Thanh Thùy,	20 văn bản, 118 Node; R@1 trên 91 câu dev 82,4% → 91,2%; κ = 1,00; phần C1–C5 chưa commit

Tuần	Công việc đã làm	Người làm	Khó khăn / kết quả
		Huỳnh Thanh Tiến	

A.1 Tuần 5 (05–09/10/2026)

A.1.1 Công việc đã làm

- **Hạ tầng**: scripts/setup_server_v2.sh, scripts/colab_server.py — chạy Ollama và model server trên Colab, mở URL công khai qua Cloudflare tunnel, in sẵn các dòng .env cho máy local.
- **Truy hồi**: hybrid BM25 + vector (src/bm25.py, QueryFusionRetriever); đo bốn cấu hình và biến thể trọng số 3:1; baseline BM25 / không xếp hạng; khoảng tin cậy bootstrap và kiểm định McNemar (eval/metrics.py).
- **Kiểm thử**: 26 kiểm thử offline trong tests/, CI GitHub Actions, Makefile.
- **Giao diện**: Gradio (app/app.py) ghi mỗi câu hỏi vào eval/results/questions_log.jsonl.
- **Đánh giá câu trả lời**: chạy lại 56 câu bằng eval/run_batch_cau_hoi.py với prompt mới, chấm tay (Mục 10.2.2.4).
- **Tầng sinh**: thêm chú giải thang điểm hệ 10 / hệ 4 vào QA_PROMPT; thêm công tắc RAG_THINKING (Mục 4.5.2).
- **Báo cáo**: cập nhật Chương 3–5 và phụ lục này.

A.1.2 Kết quả và khó khăn

- Truy hồi (vector): bộ 01 R@1 81%, R@3 100%; bộ 02 R@1 90%. Đo lại ngày 08/10 cho cùng chỉ số.
- Rerank và hybrid không cải thiện trên 54 Node, nên để mặc định tắt; tuần 6 đổi kết luận sau khi tìm ra lỗi reranker cắt ở 512 token (Chương 8).
- Phiên chạy lại 56 câu: tỷ lệ "không tìm thấy" giảm từ 52% xuống 41%, nhưng vẫn sai ở số tuần của học kỳ chính, điểm F và câu hỏi về chứng chỉ tiếng Anh; một câu trả về chuỗi /no_think.

- Lỗi mới phát hiện: nhầm thang điểm 10 với thang điểm 4 ("A được quy đổi thành 4,0 điểm hệ 10"), và câu trả lời về điểm không phân biệt khóa tuyển sinh trước 2025 với từ 2025 trở đi (QĐ 3183).
- Khi đối chiếu lại khoản 4 Điều 9, nhóm phát hiện đáp án tham chiếu cũ của câu "học bị F" bị nhầm; đã sửa trong Bảng 10.1.

A.1.3 Việc chuyển sang tuần 6

1. Chạy make eval-answers trên LLM Colab; chạy lại 56 câu theo từng yếu tố (prompt, top-*k* đưa vào LLM, RAG_THINKING) để đo tác dụng của chú giải thang điểm.
2. Thêm metadata khóa áp dụng cho mục sửa đổi của QĐ 3183 và yêu cầu câu trả lời nêu khóa.
3. Lọc chuỗi /no_think khỏi đầu ra và đếm như một lỗi khi chấm.
4. Nạp văn bản quy đổi chứng chỉ tiếng Anh (thư mục 2_quy_doi_tieng_anh): câu "quy định về tiếng anh đầu ra" trong nhật ký Gradio hiện trả lời "không tìm thấy".

A.2 Tuần 6 (12/10–16/10/2026)

A.2.1 Công việc đã làm

- **Phan Hữu Kha** (nhánh khaph, PR #18): xác minh và điền metadata hiệu lực cho mọi văn bản đang nạp, ghi nguồn xác minh trong _bao_cao_kiem_tra.md của từng thư mục (D1, commit 50e73f0); nạp đợt 3 gồm hướng dẫn phúc khảo, quy định tiết học và phòng học, thông báo chuẩn đầu ra 2020 (D2, bfe96f9); chép 5 bảng quy đổi chứng chỉ tiếng Anh ra CSV (D3, 7bf3a27); viết 15 câu dev đợt 3 (D5, a54ea28); thay placeholder Điều 9 và bổ sung minh chứng chương 7, 10 trong bản LaTeX (R6, R8).
- **Võ Thị Như Ý** (nhánh nhuy, PR #16, #21): viết 40 câu tập kiểm chứng chia 6 loại câu (K1, commit 2b79aa3, 418be41); xóa file .aux cũ, sửa 4 tham chiếu chéo sai, sinh lại cây thư mục và sửa mô tả test_ocr.py (R2, R4, 835aa8d); nhật ký cá nhân tuần 6 (cede59b).
- **Trần Hà Thanh Thùy** (nhánh ththuy-K2, PR #17, #19): gán nhãn độc lập 40 câu của K1, viết 20 câu ngoài phạm vi kèm hành vi mong đợi, gán phạm vi cho 56 câu của phiên hỏi đáp cũ (K2, commit 779b163, 5747020, d4471f5).

- **Huỳnh Thanh Tiến**: reranker đọc 2 048 token (C1); lọc phiên bản src/phien_ban.py kèm kiểm thử (C2); đo chốt cấu hình 8 cấu hình trên 91 câu dev (C3); luật chấm theo số hiệu văn bản, nhiều nhãn đúng (C4); kết luận về hybrid (C5); dựng lại phần truy hồi bằng LangChain (R10); đối chiếu CSV với PDF (D4) và tính độ đồng thuận nhãn (K3); cập nhật báo cáo theo nhận xét; merge PR #16–#21. Phần mã nguồn này đang ở nhánh ttien, chưa commit.

A.2.2 Kết quả và khó khăn

- Corpus tăng từ 6 lên 20 văn bản, 118 Node; chỉ mục khớp 118/118 với loader.
- Truy hồi trên 91 câu dev: R@1 tăng từ 82,4% (vector) lên 91,2% (rerank cùng lọc phiên bản); dev đợt 2 đạt 15/15. Bộ 02 đạt 7/10, chưa tới mục tiêu 80%.
- Lần đo đầu, bước lọc phiên bản đẩy TB 2567 (chỉ bổ sung chứng chỉ PTE) lên top-1 cho câu hỏi về IELTS, VSTEP vì front matter ghi 2567 là văn bản sửa đổi; đã sửa bằng ngưỡng điểm rerank (Mục 8.3).
- Chạy liền 8 cấu hình trên GPU 4 GB bị tràn bộ nhớ vì mỗi cấu hình nạp một reranker riêng; đã giải phóng bộ nhớ giữa các cấu hình trong eval/evaluate.py.
- Đối chiếu CSV với PDF phát hiện 24 ô sai biên ở TB 2366 (bất đẳng thức chặt bị chép thành khoảng có hai đầu).
- Tập kiểm chứng: trùng 40/40 nhãn giữa hai người ($\kappa = 1{,}00$); 4 câu gần như trùng ý với câu dev.
- Sinh câu trả lời trên cấu hình chốt (sáng 17/10): 24/28 câu có đáp án đúng, 2 đúng một phần, 2 sai; mỗi câu gọi LLM một lượt, khoảng 9 giây. Cấu hình cũ chưa chạy được vì Ollama trên Colab dừng khi ngữ cảnh dài (Mục 10.2.2.5).

A.2.3 Việc chuyển sang tuần 7

1. Chạy lại cấu hình cũ và 2 câu bị lỗi kết nối bằng eval/tra_loi_dev.py để so với cấu hình chốt; chạy make test-llm; viết đáp án cho 20 câu dev đợt 1 để chấm.
2. Xử lý 4 câu kiểm chứng trùng ý với dev, đóng băng tập kiểm chứng bằng sha256 rồi chạy đúng một lần; điền cột test ở Bảng 4.3 và 8.1.
3. Commit và mở PR cho phần C1–C5, D4, K3.
4. Thống nhất cận dưới khóa áp dụng của TB 2366/2451 và quy tắc tra bảng cho các khoảng chồng biên của văn bản gốc.
5. Viết phần cá nhân: định hướng nghề nghiệp, AI Disclosure, định hướng bản thân.

PHỤ LỤC B. BẢNG TỰ KIỂM NHANH BA TẦNG

Bảng B.1: Tự kiểm ba tầng.

Tầng	Nội dung	Nhóm đã đạt
1A	Lõi LlamaIndex	6 / 6 mục (Bảng 4.1); mục 6 mới đạt ở phần đánh giá truy hồi, đánh giá câu trả lời bằng evaluator chưa chạy
1B	Tự chọn	Custom node parser theo Điều; persistent index có fingerprint; trích dẫn nguồn; nhiều nguồn (20 văn bản); từ chối ngoài corpus (một phần, chưa đo tỷ lệ đúng)
2	Chuẩn kỹ nghệ	.env/.env.example, ghim phiên bản requirements.txt, 54 kiểm thử, CI GitHub Actions, Makefile, README
3	Nâng cao	Lọc phiên bản theo metadata hiệu lực, rerank cross-encoder fp16 đọc 2 048 token, hybrid BM25 + vector (RRF), model server HTTP, đánh giá có bootstrap CI và McNemar. Điểm sâu nhất có thể giải thích và sửa tại chỗ: chưa chốt

PHỤ LỤC C. THỰC NGHIỆM HỌC TẬP

Các thí nghiệm dưới đây chỉ dùng để học hoặc để ra quyết định, không khai trong Bảng phủ kỹ thuật.

- **Hybrid ưu tiên vector 3:1** (relative_score): tốt nhất trên bộ 01 (R@1 84%) nhưng kém vector trên bộ 02; kết quả ở eval/results/truy-hoi-hybrid-w.md.
- **Hai mô hình embedding**: profile local (vietnamese-bi-encoder, 768 chiều, nhận tối đa 256 token, cần tách từ) cho Recall thấp hơn hẳn profile server. Số đo cũ (R@1 64,5% trên 31 câu) thuộc pipeline một văn bản 21 Node nên không so trực tiếp được với số hiện tại. *Trên corpus hiện tại, 63/118 Node dài hơn 256 token nên mất phần cuối khi embed; kết quả profile local vì vậy chỉ để tham khảo, không dùng để báo cáo chất lượng.*
- **Chất lượng OCR**: tỷ lệ âm tiết tiếng Việt hợp lệ dùng để chọn giữa các bản scan; QĐ 3183/2025 thấp nhất (94,9%).